Yolov11原理和代码详解

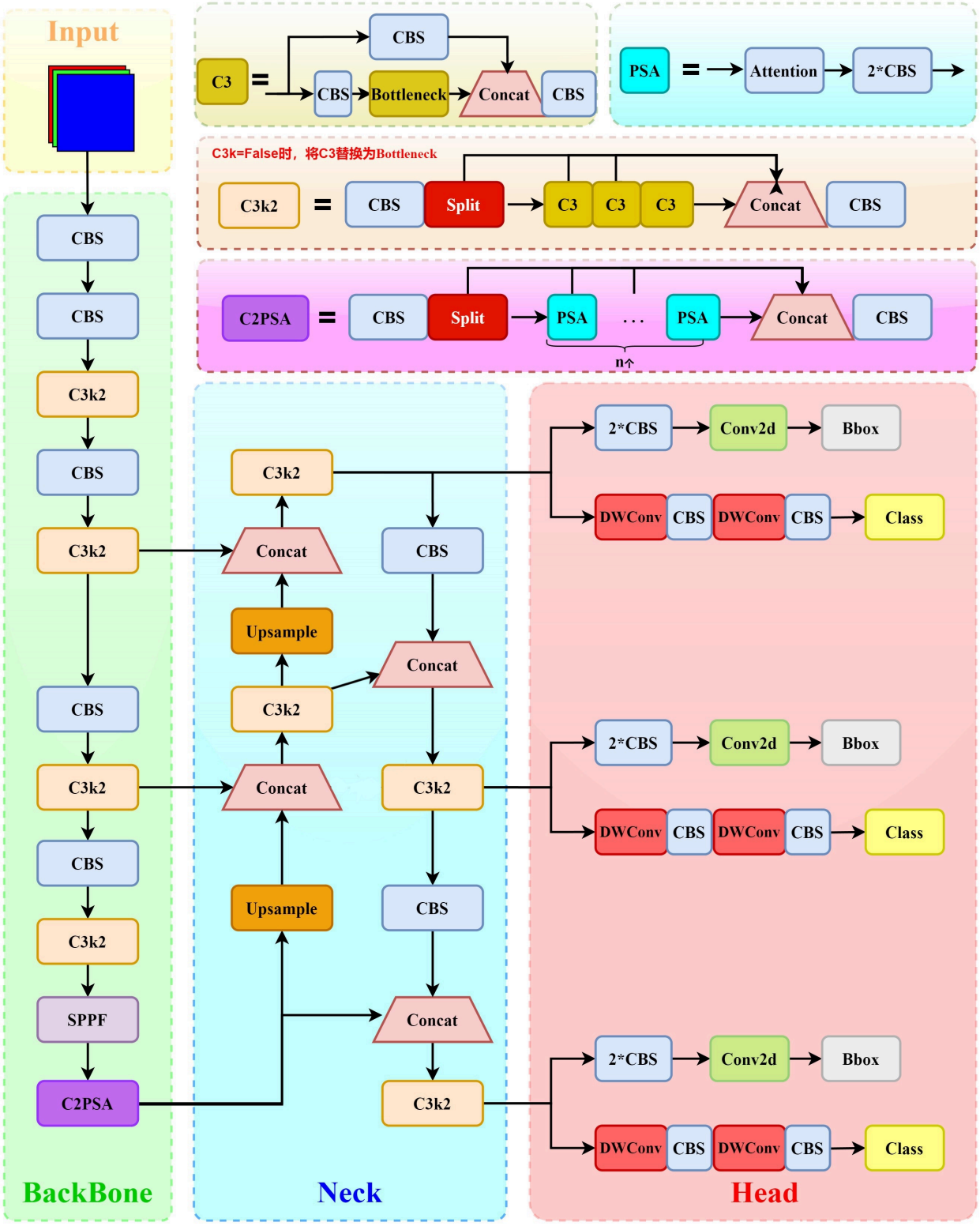
1.yoloV11架构

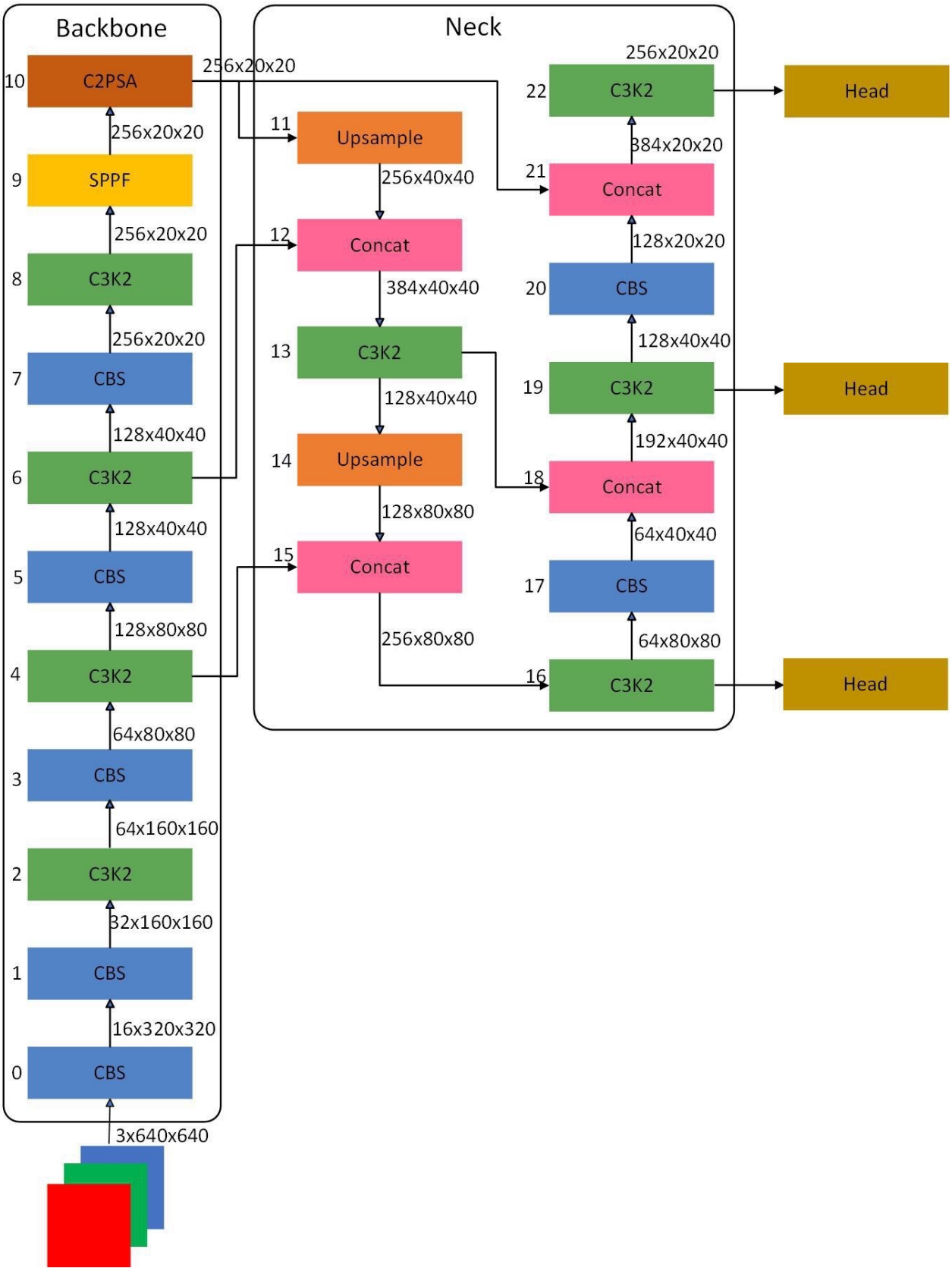
1.1 模型介绍

代码地址：https://github.com/ultralytics/ultralytics

YOLOv11继承自YOLOv8，在YOLOv8基础上进行了改进，使同等精度下参数量降低20%，在速度和准确性方面具有无与伦比的性能。其流线型设计使其适用于各种应用，并可轻松适应从边缘设备到云 API 等不同硬件平台。使其成为各种物体检测与跟踪、实例分割、图像分类和姿态估计任务的绝佳选择。

1.2 架构图


Input
C3
=
CBS
CBS
Bottleneck
Concat
CBS
PSA
=
Attention
2*CBS
C3k=False时，将C3替换为Bottleneck
C3k2
=
CBS
Split
C3
C3
C3
Concat
CBS
C2PSA
=
CBS
Split
PSA
...
PSA
n个
Concat
CBS
CBS
CBS
C3k2
CBS
C3k2
CBS
C3k2
CBS
C3k2
SPPF
C2PSA
BackBone
C3k2
Concat
Upsample
C3k2
Concat
Upsample
CBS
Concat
C3k2
CBS
Concat
C3k2
Neck
2*CBS
Conv2d
Bbox
DWConv
CBS
DWConv
CBS
Class
2*CBS
Conv2d
Bbox
DWConv
CBS
DWConv
CBS
Class
2*CBS
Conv2d
Bbox
DWConv
CBS
DWConv
CBS
Class
Head



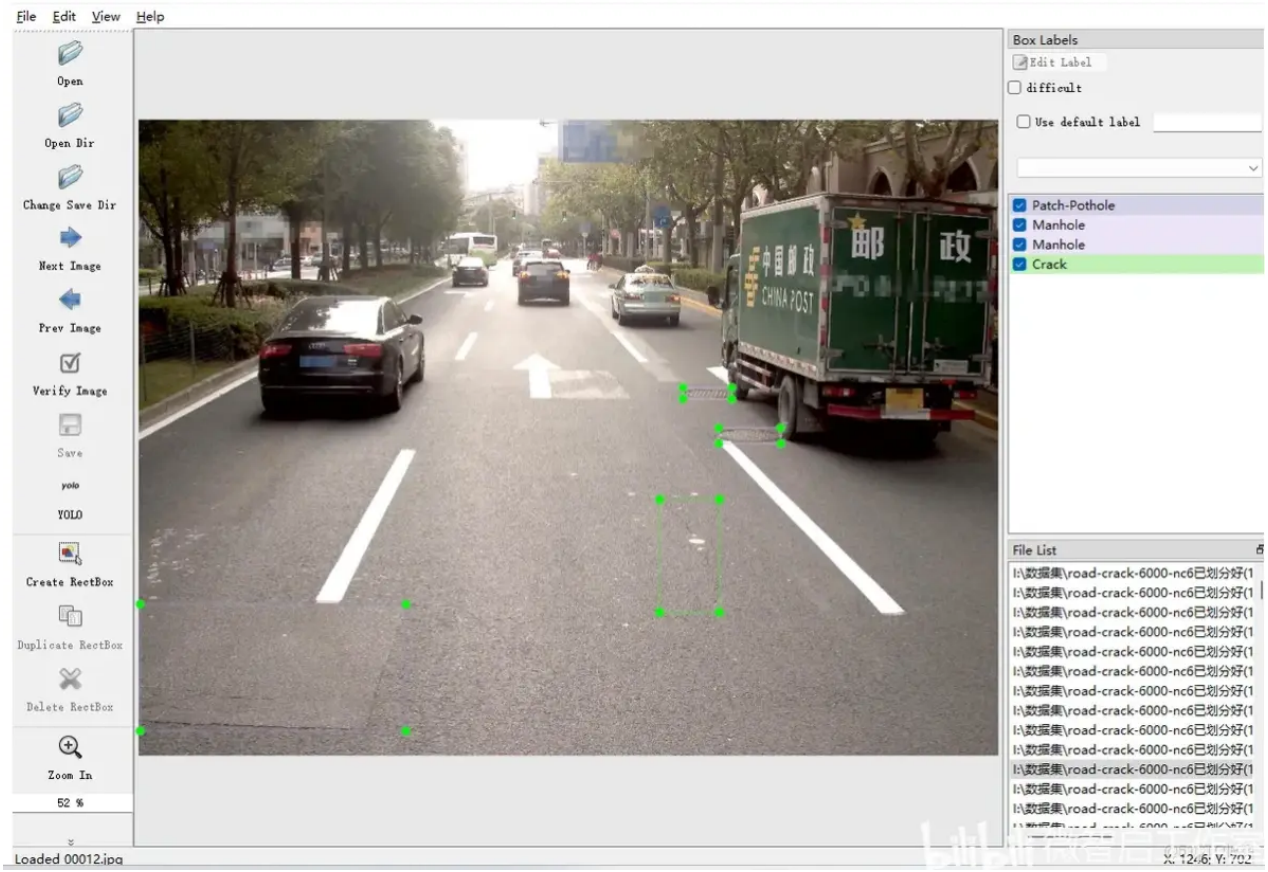
2. 项目准备

2.1 环境准备

- 首先创建虚拟环境
- 然后 pip install ultralytics 会自动安装所需要的包

2.2 数据介绍

路面裂缝、井盖、坑洼路面的标识数据集，使用labelimg标注，标签的格式是txt格式，适用于yolo目标检测系列所有版本训练数据集。



数据集总共有9个类，已经划分好训练集和验证集。

训练的配置参数信息如下

> nc: 9

> names: ["Crack", "Manhole", "Net", "Pothole", "Patch-Crack", "Patch-Net", "Patch-Pothole", "other", "Other"]

下载内容

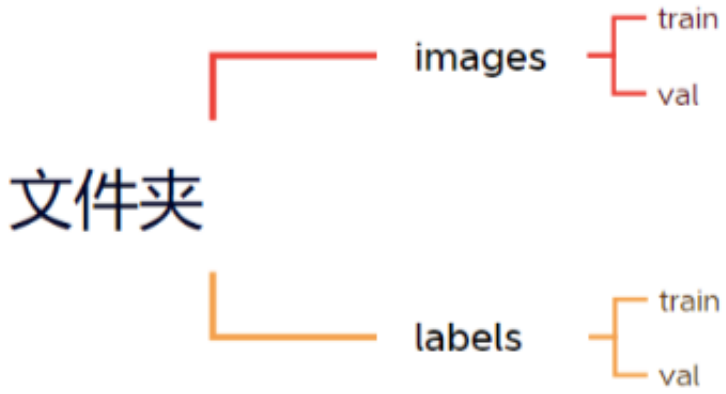
文件大小：2.47G

下载方式：百度网盘 链接: https://pan.baidu.com/s/1T9mc8zzaqSooaRzq_BXecg?pwd=96xm 提取码: 96xm

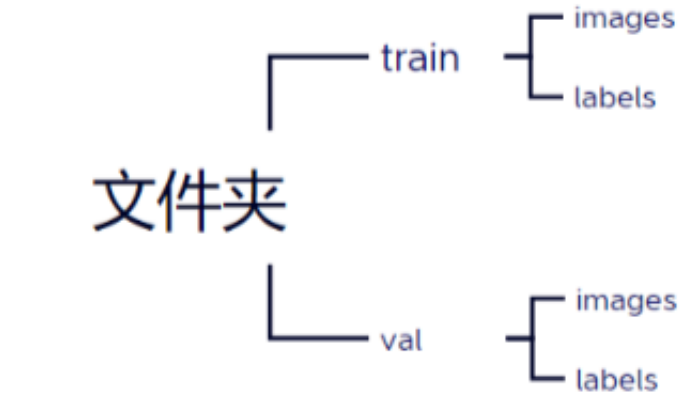
2.3 数据准备

数据集有以下两种方式放置，都可以进行训练，常见的数据集放置是第一种。

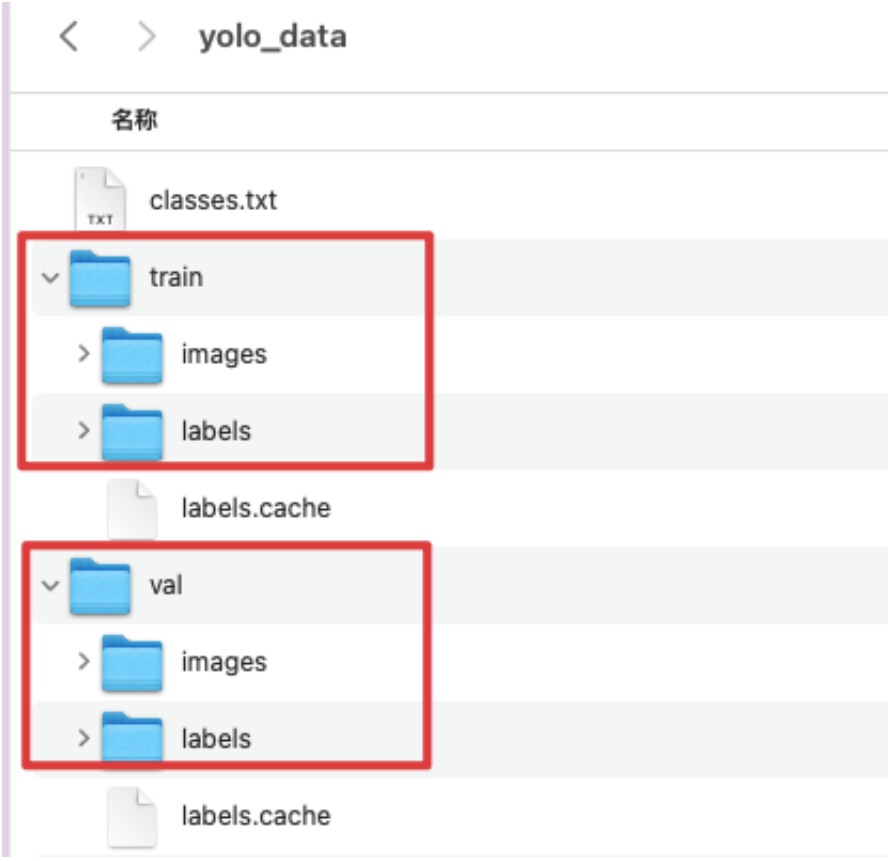
方式一：



方式二：



本数据集按照第二种方式放置：



2.4 下载代码

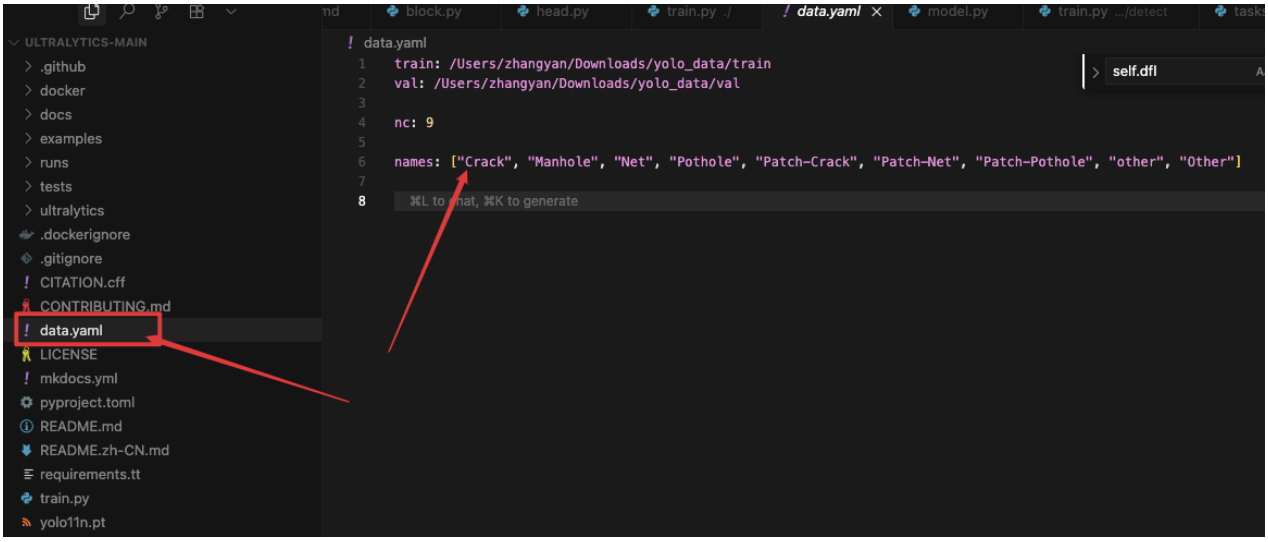
官方代码地址：https://github.com/ultralytics/ultralytics

本项目代码地址: https://pan.baidu.com/s/1Pnun3TeQ45lBrjvQSPLIQg?pwd=6389

2.5 构造data.yaml

在代码根目录下新建文件`data.yaml`,并编写如下内容：

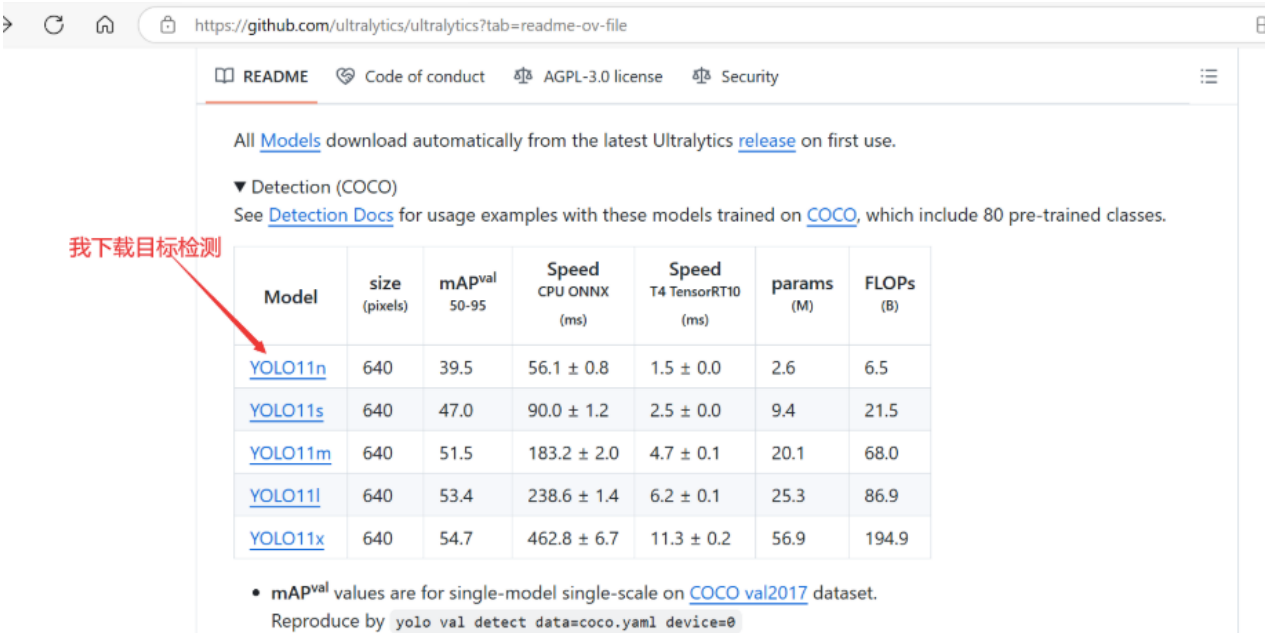
```yaml
train: /Users/zhangyan/Downloads/yolo_data/train #此处修改为数据实际目录
val: /Users/zhangyan/Downloads/yolo_data/val #此处修改为数据实际目录

nc: 9

names: ["Crack", "Manhole", "Net", "Pothole", "Patch-Crack", "Patch-Net",
"Patch-Pothole", "other", "Other"]
```



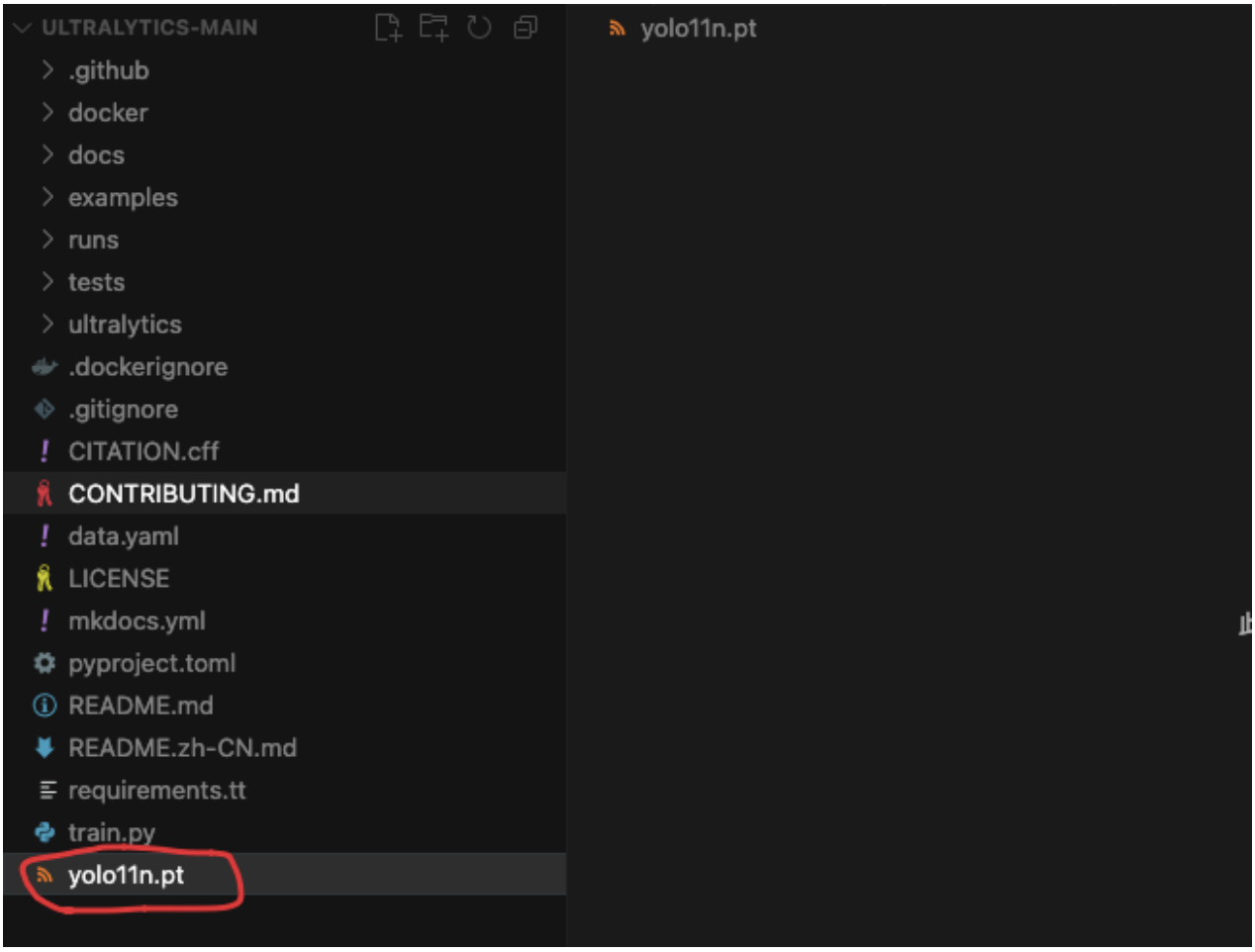
2.6 下载模型

为了方便调试运行，本项目基于yolov11 最小的版本yolov11n进行微调。

模型下载地址：https://github.com/ultralytics/ultralytics，



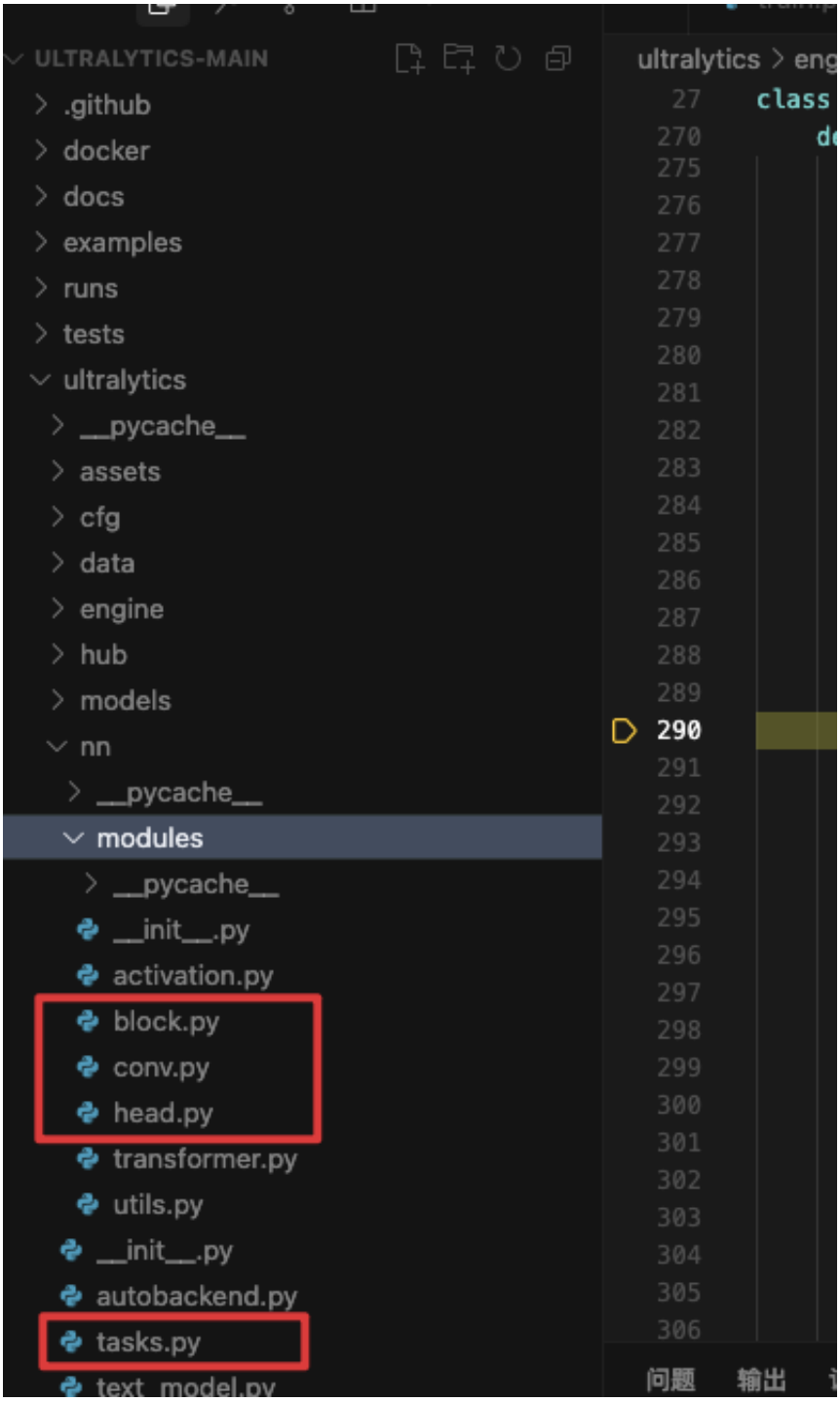
下载后将`yolo11n.pt`放入项目根目录：



3. 代码详解

3.1 代码结构

有三部分比较重要的代码：conv、block、tasks



conv.py

conv定义了一些基础的卷积算子结构

```
from .conv import (
    CBAM,
    ChannelAttention,
    Concat,
    Conv,
    Conv2,
    ConvTranspose,
    DWConv,
    DWConvTranspose2d,
    Focus,
    GhostConv,
    Index,
    LightConv,
    RepConv,
    SpatialAttention,
)
```

以最简单的卷积模块为例，他就是把`卷积层+归一化层+激活层`合在一起，然后在block中调用：

```python
class Conv(nn.Module):
    """
    Standard convolution module with batch normalization and activation.
    标准卷积模块，包含BN和激活函数。"""

    default_act = nn.SiLU()  # default activation

    def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
        """Initialize Conv layer with given parameters."""
        super().__init__()
        self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d), groups=g, dilation=d, bias=False)
        self.bn = nn.BatchNorm2d(c2)
        self.act = self.default_act if act is True else act if isinstance(act, nn.Module) else nn.Identity()

    def forward(self, x):
        return self.act(self.bn(self.conv(x)))

    def forward_fuse(self, x):
        """Apply convolution and activation without batch normalization."""
        return self.act(self.conv(x))
```

block.py

在block中定义了构成网络的模块形式

```python
__all__ = (
    "DFL",
    "HGBlock",
```

```
    "HGStem",
    "SPP",
    "SPPF",
    "C1",
    "C2",
    "C3",
    "C2f",
    "C2fAttn",
    "ImagePoolingAttn",
    "ContrastiveHead",
    "BNContrastiveHead",
    "C3x",
    "C3TR",
    "C3Ghost",
    "GhostBottleneck",
    "Bottleneck",
    "BottleneckCSP",
    "Proto",
    "RepC3",
    "ResNetLayer",
    "RepNCSPELAN4",
    "ELAN1",
    "ADown",
    "AConv",
    "SPPELAN",
    "CBFuse",
    "CBLinear",
    "C3k2",
    "C2fPSA",
    "C2PSA",
    "RepVGGDW",
    "CIB",
    "C2fCIB",
    "Attention",
    "PSA",
    "SCDown",
    "TorchVision",
)
```

最大创新点c3k2就是在这里定义的:

```
class C3k2(C2f):
    """
    Faster Implementation of CSP Bottleneck with 2 convolutions.

    可选C3k块的C2f结构。
    """

    def __init__(
```

```
        self, c1: int, c2: int, n: int = 1, c3k: bool = False, e: float =
0.5, g: int = 1, shortcut: bool = True
    ):
        """
        Initialize C3k2 module.

        初始化C3k2结构。

        Args:
            c1 (int): Input channels. 输入通道数
            c2 (int): Output channels. 输出通道数
            n (int): Number of blocks. 块数
            c3k (bool): Whether to use C3k blocks. 是否用C3k
            e (float): Expansion ratio. 通道扩展比例
            g (int): Groups for convolutions. 分组数
            shortcut (bool): Whether to use shortcut connections. 是否使用
残差
        """
        super().__init__(c1, c2, n, shortcut, g, e)
        self.m = nn.ModuleList(
            C3k(self.c, self.c, 2, shortcut, g) if c3k else
Bottleneck(self.c, self.c, shortcut, g) for _ in range(n)
        )
```

3.2 编写train.py

在项目根目录中新建文件`train.py`,编写如下代码:

```
from ultralytics import YOLO
# 加载官方预训练模型
model = YOLO("yolo11n.pt",task="detect")  #此行打断点
# 模型训练
results = model.train(data="data.yaml", epochs=100, batch=4)
```

在实例化模型的代码行打断点，进行调试

训练参数说明

YOLOv11 模型的训练设置包括训练过程中使用的各种超参数和配置。这些设置会影响模型的性能、速度和准确性。关键的训练设置包括批量大小、学习率、动量和权重衰减。此外，优化器、损失函数和训练数据集组成的选择也会影响训练过程。对这些设置进行仔细的调整和实验对于优化性能至关重要。以下是官方给出了训练可设置参数和说明:

参数	默认值	说明
`model`	`None`	指定用于训练的模型文件。接受指向 `.pt` 预训练模型或 `.yaml` 配置文件。对于定义模型结构或初始化权重至关重要。

`data`	`None`	数据集配置文件的路径（例如 `coco8.yaml`).该文件包含特定于数据集的参数，包括训练数据和验证数据的路径、类名和类数。
`epochs`	`100`	训练总轮数。每个epoch代表对整个数据集进行一次完整的训练。调整该值会影响训练时间和模型性能。
`time`	`None`	最长训练时间（小时）。如果设置了该值，则会覆盖 `epochs` 参数，允许训练在指定的持续时间后自动停止。对于时间有限的训练场景非常有用。
`patience`	`100`	在验证指标没有改善的情况下，提前停止训练所需的epoch数。当性能趋于平稳时停止训练，有助于防止过度拟合。
`batch`	`16`	批量大小，有三种模式:设置为整数(例如，' Batch =16 '), 60% GPU内存利用率的自动模式(' Batch =-1 '），或指定利用率分数的自动模式(' Batch =0.70 ')。
`imgsz`	`640`	用于训练的目标图像尺寸。所有图像在输入模型前都会被调整到这一尺寸。影响模型精度和计算复杂度。
`save`	`True`	可保存训练检查点和最终模型权重。这对恢复训练或模型部署非常有用。
`save_period`	`-1`	保存模型检查点的频率，以 epochs 为单位。值为-1 时将禁用此功能。该功能适用于在长时间训练过程中保存临时模型。
`cache`	`False`	在内存中缓存数据集图像（`True` / `ram` ）、磁盘（`disk` ），或禁用它（`False`).通过减少磁盘 I/O 提高训练速度，但代价是增加内存使用量。
`device`	`None`	指定用于训练的计算设备：单个 GPU（`device=0` ）、多个 GPU（`device=0,1`)、CPU（`device=cpu`），或苹果芯片的 MPS（`device=mps`).
`workers`	`8`	加载数据的工作线程数（每 `RANK` 多 GPU 训练）。影响数据预处理和输入模型的速度，尤其适用于多 GPU 设置。
`project`	`None`	保存训练结果的项目目录名称。允许有组织地存储不同的实验。
`name`	`None`	训练运行的名称。用于在项目文件夹内创建一个子目录，用于存储训练日志和输出结果。
`exist_ok`	`False`	如果为 True，则允许覆盖现有的项目/名称目录。这对迭代实验非常有用，无需手动清除之前的输出。
`pretrained`	`True`	决定是否从预处理模型开始训练。可以是布尔值，也可以是加载权重的特定模型的字符串路径。提高训练效率和模型性能。

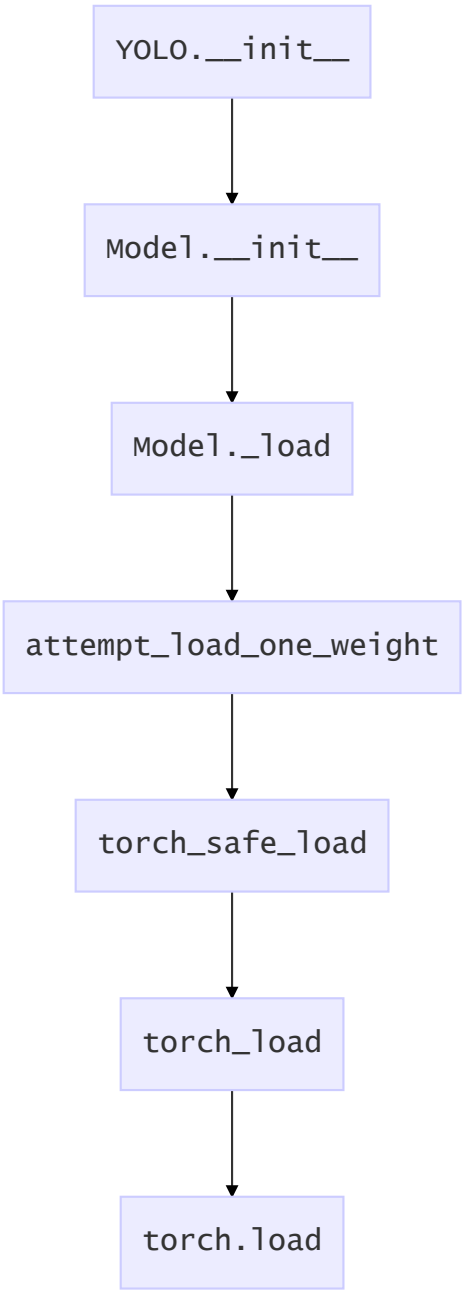
`optimizer`	`'auto'`	为训练模型选择优化器。选项包括 `SGD`, `Adam`, `AdamW`, `NAdam`, `RAdam`, `RMSProp` 等，或 `auto` 用于根据模型配置进行自动选择。影响收敛速度和稳定性
`verbose`	`False`	在训练过程中启用冗长输出，提供详细日志和进度更新。有助于调试和密切监控培训过程。
`seed`	`0`	为训练设置随机种子，确保在相同配置下运行的结果具有可重复性。
`deterministic`	`True`	强制使用确定性算法，确保可重复性，但由于对非确定性算法的限制，可能会影响性能和速度。
`single_cls`	`False`	在训练过程中将多类数据集中的所有类别视为单一类别。适用于二元分类任务，或侧重于对象的存在而非分类。
`rect`	`False`	可进行矩形训练，优化批次组成以减少填充。这可以提高效率和速度，但可能会影响模型的准确性。
`cos_lr`	`False`	利用余弦学习率调度器，根据历时的余弦曲线调整学习率。这有助于管理学习率，实现更好的收敛。
`close_mosaic`	`10`	在训练完成前禁用最后 N 个epoch的马赛克数据增强以稳定训练。设置为 0 则禁用此功能。
`resume`	`False`	从上次保存的检查点恢复训练。自动加载模型权重、优化器状态和历时计数，无缝继续训练。
`amp`	`True`	启用自动混合精度 (AMP) 训练，可减少内存使用量并加快训练速度，同时将对精度的影响降至最低。
`fraction`	`1.0`	指定用于训练的数据集的部分。允许在完整数据集的子集上进行训练，这对实验或资源有限的情况非常有用。
`profile`	`False`	在训练过程中，可对ONNX 和TensorRT 速度进行剖析，有助于优化模型部署。
`freeze`	`None`	冻结模型的前 N 层或按索引指定的层，从而减少可训练参数的数量。这对微调或迁移学习非常有用。
`lr0`	`0.01`	初始学习率（即 `SGD=1E-2`, `Adam=1E-3`）.调整这个值对优化过程至关重要，会影响模型权重的更新速度。
`lrf`	`0.01`	最终学习率占初始学习率的百分比 = (`lr0 * lrf`)，与调度程序结合使用，随着时间的推移调整学习率。
`momentum`	`0.937`	用于 SGD 的动量因子，或用于 Adam 优化器的 beta1，用于将过去的梯度纳入当前更新。
`weight_decay`	`0.0005`	L2 正则化项，对大权重进行惩罚，以防止过度拟合。

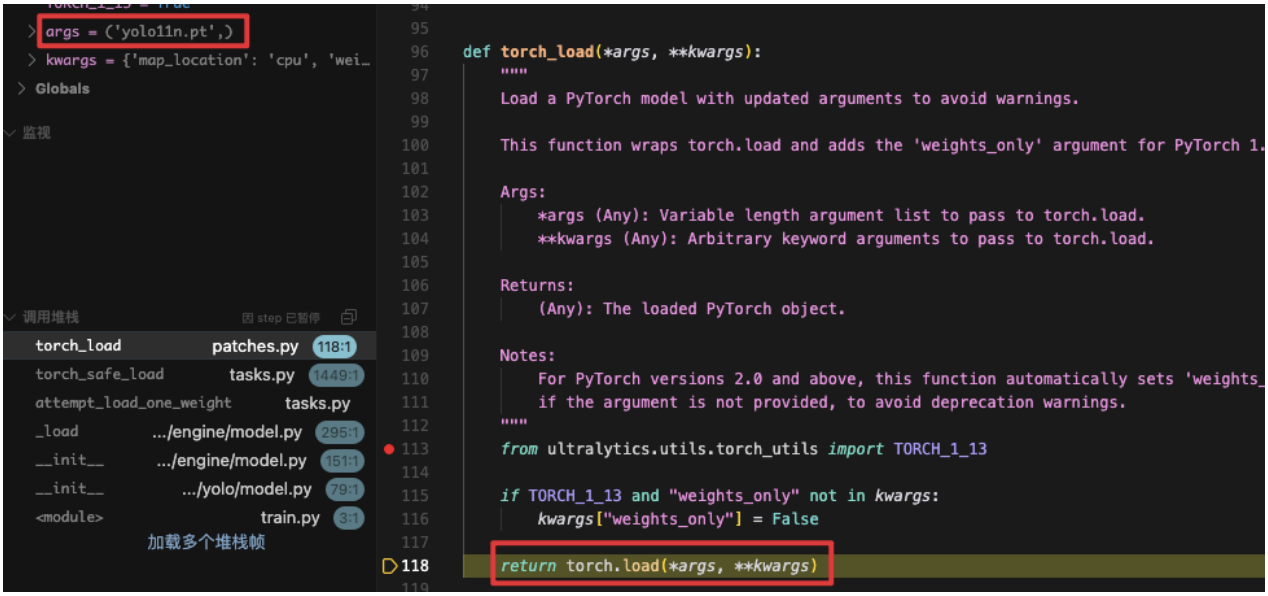
warmup_epochs	3.0	学习率预热的历元数，学习率从低值逐渐增加到初始学习率，以在早期稳定训练。
warmup_momentum	0.8	热身阶段的初始动力，在热身期间逐渐调整到设定动力。
warmup_bias_lr	0.1	热身阶段的偏置参数学习率，有助于稳定初始历元的模型训练。
box	7.5	损失函数中边框损失部分的权重，影响对准确预测边框坐标的重视程度。
cls	0.5	分类损失在总损失函数中的权重，影响正确分类预测相对于其他部分的重要性。
dfl	1.5	分布焦点损失权重，在某些YOLO 版本中用于精细分类。
pose	12.0	姿态损失在姿态估计模型中的权重，影响着准确预测姿态关键点的重点。
kobj	2.0	姿态估计模型中关键点对象性损失的权重，平衡检测可信度与姿态精度。
label_smoothing	0.0	应用标签平滑，将硬标签软化为目标标签和标签均匀分布的混合标签，可以提高泛化效果。
nbs	64	用于损耗正常化的标称批量大小。
overlap_mask	True	决定在训练过程中分割掩码是否应该重叠，适用于实例分割任务。
mask_ratio	4	分割掩码的下采样率，影响训练时使用的掩码分辨率。
dropout	0.0	分类任务中正则化的丢弃率，通过在训练过程中随机省略单元来防止过拟合。
val	True	可在训练过程中进行验证，以便在单独的数据集上对模型性能进行定期评估。
plots	False	生成并保存训练和验证指标图以及预测示例图，以便直观地了解模型性能和学习进度。

常用的几个训练参数是: 数据集配置文件data、训练轮数epochs、训练批次大小batch、训练使用的设备device、模型优化器optimizer、初始学习率lr0。

3.3 模型加载

调用链：

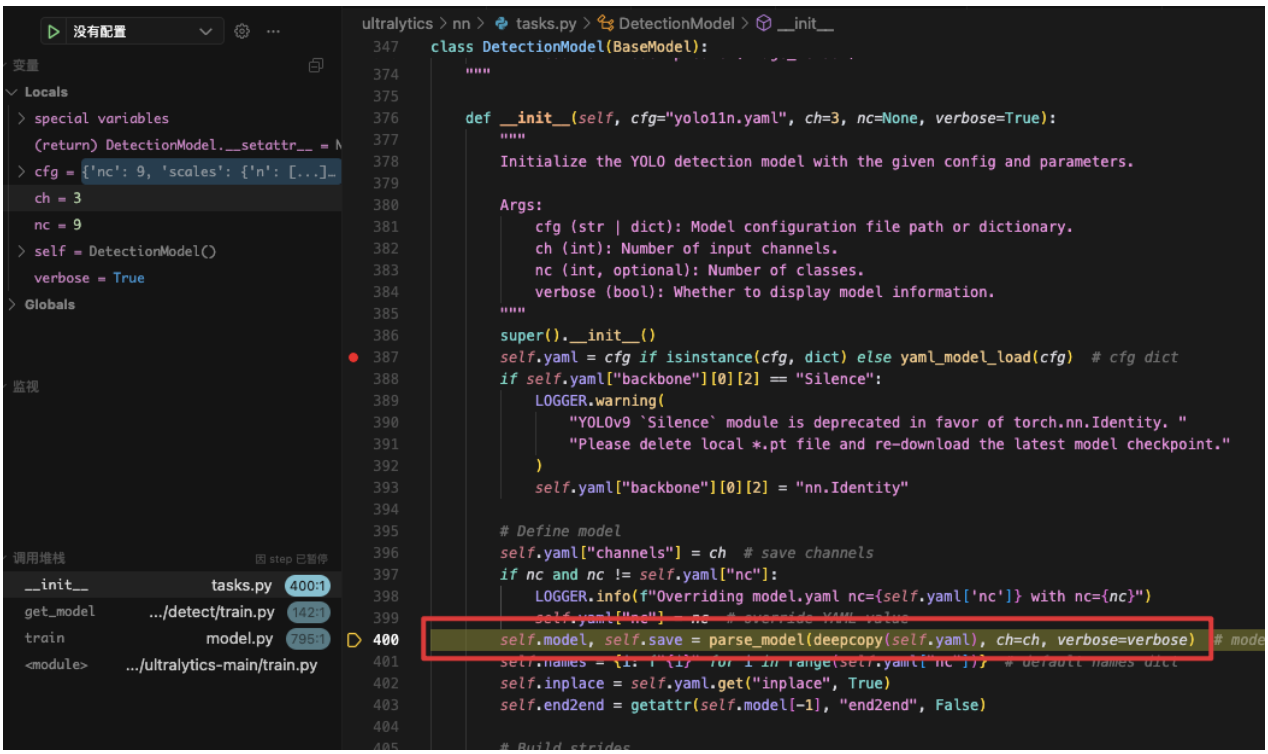

YOLO.__init__
Model.__init__
Model._load
attempt_load_one_weight
torch_safe_load
torch_load
torch.load

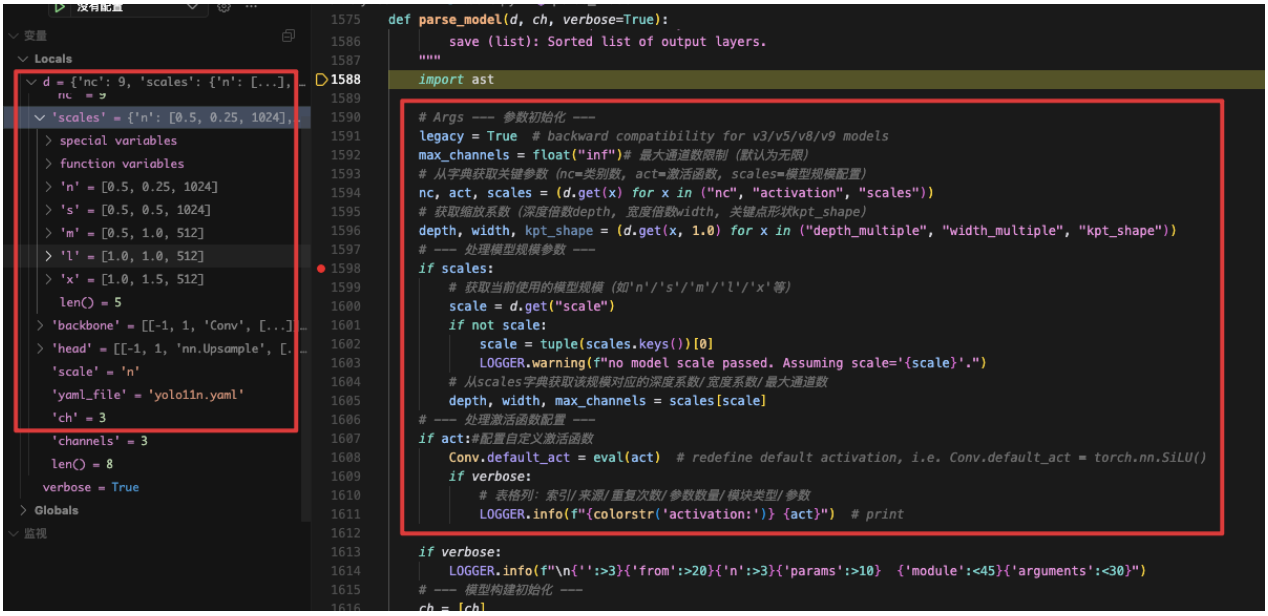


3.4 模型构建

3.4.1 parse_model

Model.train—>Model.trainer[DetectionTrainer].get_model—>DetectionModel.**init**—>**parse_model**





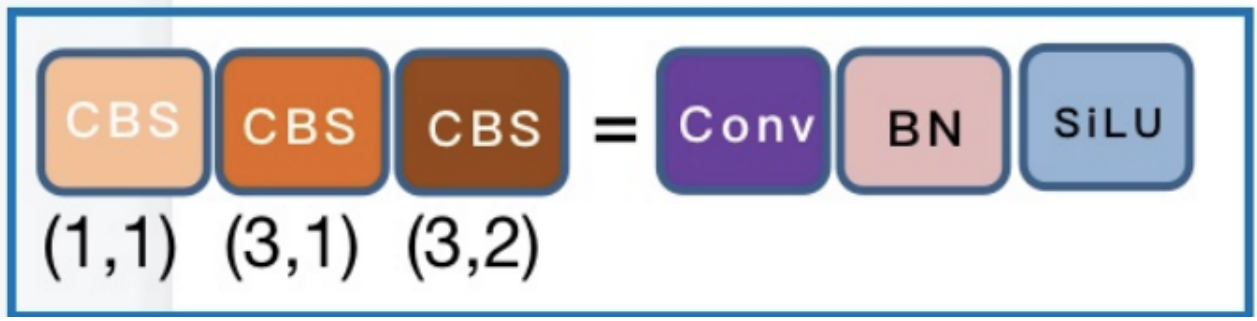
- nc: 9 指的是数据集中的类别数量。
- scales: 代表模型尺寸，分了n,s,m,l,x这5个不同大小的尺寸，参数量依次从小到大。
-
- depth深度因子的作用：表示模型中重复模块的数量或层数的缩放比例。这里主要用来调整C3k2模块中的子模块Bottelneck重复次数。比如主干中第一个C3k2模块的number系数是3，我们使用0.5x3并且向上取整就等于1了，这就代表第一个C3k2模块中Bottelneck只重复一次；
- width宽度因子的作用：表示模型中通道数（即特征图的深度）的缩放比例,如果某个层原本有64个通道，而width设置为0.5，则该层的通道数变为32。比如使用yolov11n.yaml文件，参数为[0.50, 0.25, 1024]。第一个Conv模块的输出通道数写的是64，但是实际上这个通道数并不是64，而是使用宽度因子 0.25x64得到的最终结果16；同理，C3k2模块的输出通道虽然在yaml文件上写的是128，但是在实际使用时依然要乘上宽度因子0.25，那么第一个C3k2模块最终的到实际通道数就是0.25x128 = 32。如下图所示，其他的依次类推。
- max-channels: 表示每层最大通道数。每层的通道数会与这个参数进行一个对比，如果特征图通道数大于这个数，那就取 max_channels的值。

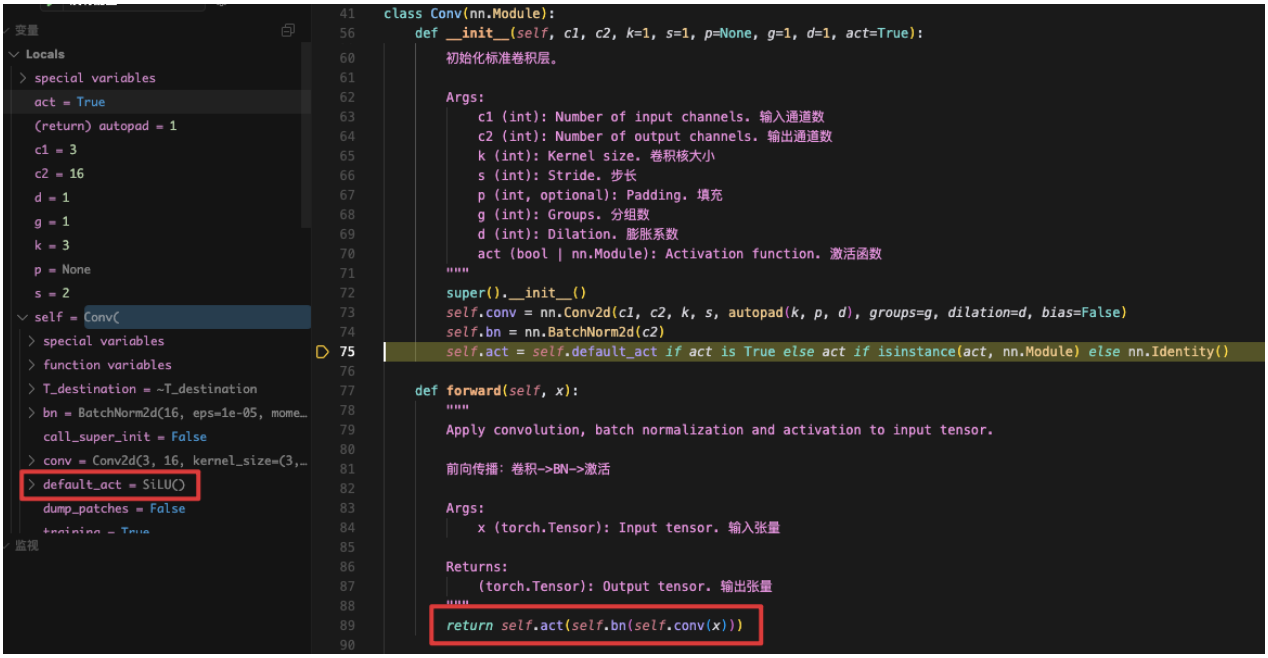
模型构建的核心代码：

```python
    # --- 遍历所有层配置（骨干网络 + 检测头）---
    # 每层格式：[来源, 重复次数, 模块名称, 参数列表]
    for i, (f, n, m, args) in enumerate(d["backbone"] + d["head"]):  # from, number, module, args
        # 解析模块类型（支持三种来源：torch.nn / torchvision.ops / 自定义模块）
        m = (
            getattr(torch.nn, m[3:])
            if "nn." in m# torch.nn中的模块（例如nn.Conv2d）
            else getattr(__import__("torchvision").ops, m[16:])
            if "torchvision.ops." in m# torchvision中的模块
            else globals()[m]# 自定义模块（从全局命名空间获取）
        )  # get module
        # 处理参数中的字符串（替换局部变量或转换为字面值）
        for j, a in enumerate(args):
            if isinstance(a, str):
                with contextlib.suppress(ValueError):
                    # 尝试在局部变量中查找 找不到就保留原始字符串
                    args[j] = locals()[a] if a in locals() else ast.literal_eval(a)
        # 计算实际重复次数（考虑深度缩放系数）
        n = n_ = max(round(n * depth), 1) if n > 1 else n  # depth gain
        # --- 根据模块类型进行特殊处理 ---
        # 处理基础模块（需要通道数计算）
        if m in base_modules:
            # c1=输入通道, c2=输出通道（来自参数第一个元素）
            c1, c2 = ch[f], args[0]
            # 输出通道调整规则
            if c2 != nc:  # if c2 not equal to number of classes (i.e. for Classify() output)
                # 1. 限制最大通道数
                # 2. 应用宽度倍数
                # 3. 按8的倍数取整（硬件优化）
                c2 = make_divisible(min(c2, max_channels) * width, 8)
            # C2fAttn模块特殊处理（嵌入维度和注意力头数）
            if m is C2fAttn:  # set 1) embed channels and 2) num heads
```

```python
                c2 = ch[f]# 输出通道=上一层的输入通道
            # --- 模块实例化 ---
            # 处理模块重复构建（n>1时创建序列结构）
            m_ = torch.nn.Sequential(*(m(*args) for _ in range(n))) if n > 1 else m(*args)  # module
            # 获取模块类型名称（简化字符串）
            t = str(m)[8:-2].replace("__main__.", "")  # module type
            m_.np = sum(x.numel() for x in m_.parameters())  # number params  计算参数数量
            m_.i, m_.f, m_.type = i, f, t  # attach index, 'from' index, type 附加元信息：当前索引/来源索引/模块类型
            if verbose:
                LOGGER.info(f"{i:>3}{str(f):>20}{n_:>3}{m_.np:10.0f}  {t:<45}{str(args):<30}")  # print
            save.extend(x % i for x in ([f] if isinstance(f, int) else f) if x != -1)  # append to savelist
            layers.append(m_)
            if i == 0:# 更新通道记录（首层初始化ch列表）
                ch = []
            ch.append(c2)
        # 返回Sequential模型和排序后的输出层索引
        return torch.nn.Sequential(*layers), sorted(save)
```

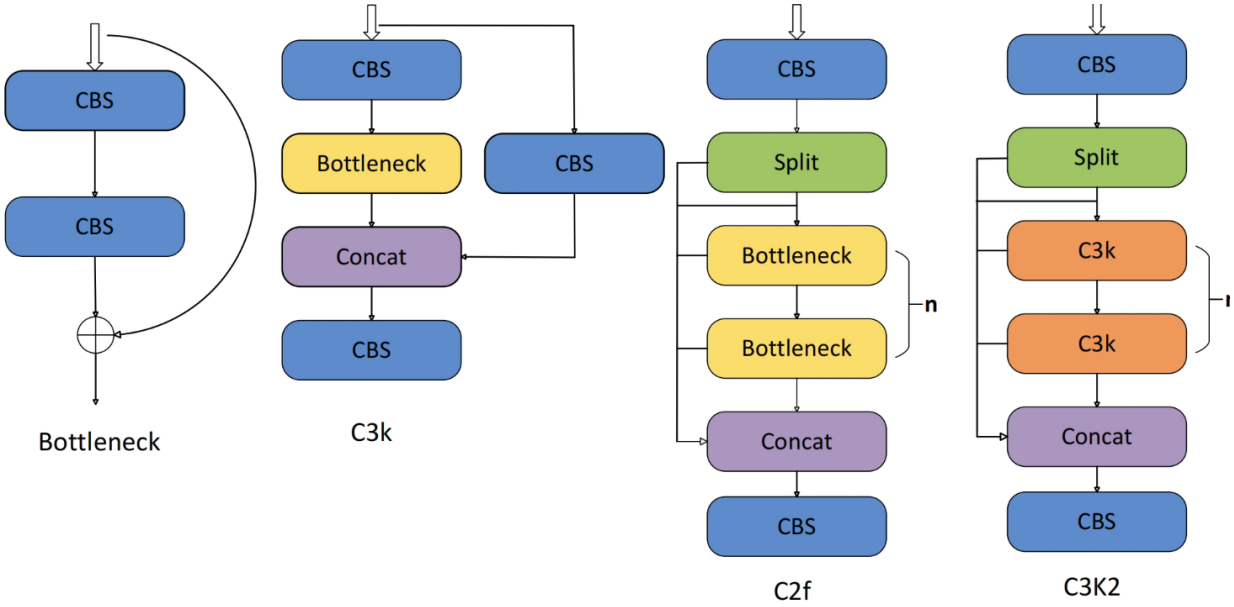
3.4.2 CBS



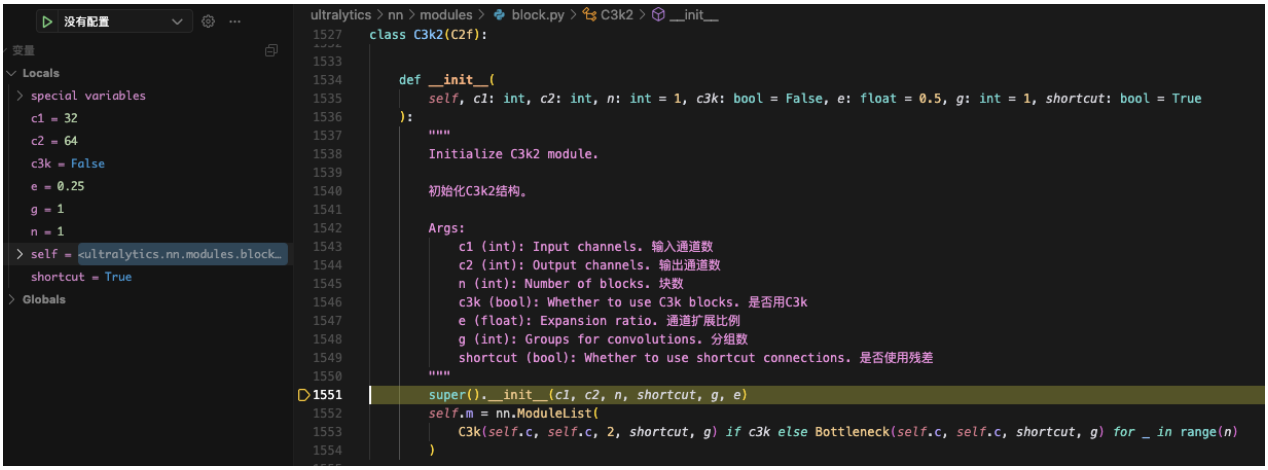


3.4.3 C3k2

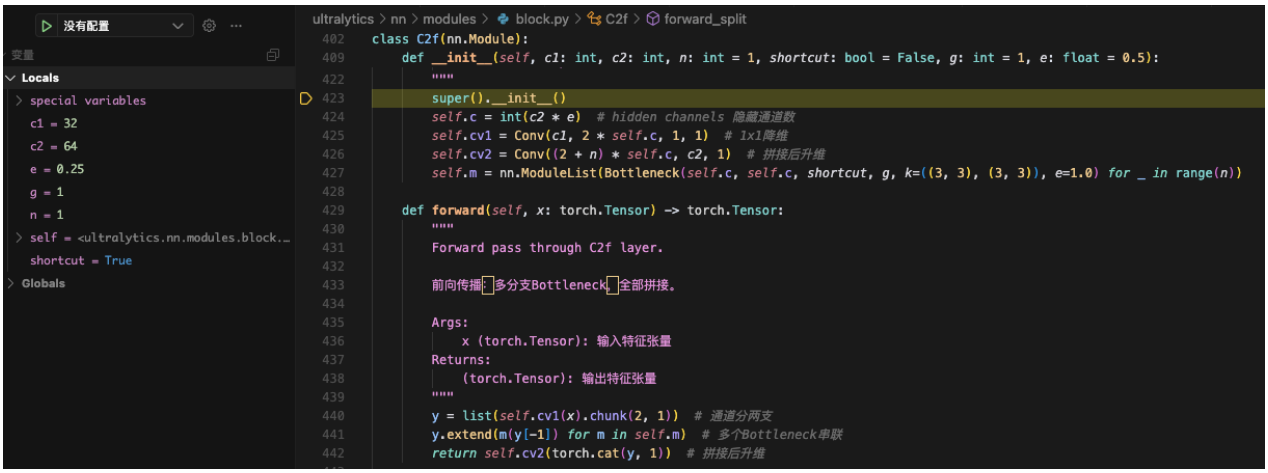
从下面图中我们可以看到，C3K2模块其实就是C2F模块转变出来的，它代码中有一个设置，就是当c3k这个参数为FALSE的时候，C3K2模块就是C2F模块，也就是说它的Bottleneck是普通的Bottleneck；反之当它为true的时候，将Bottleneck模块替换成C3模块。



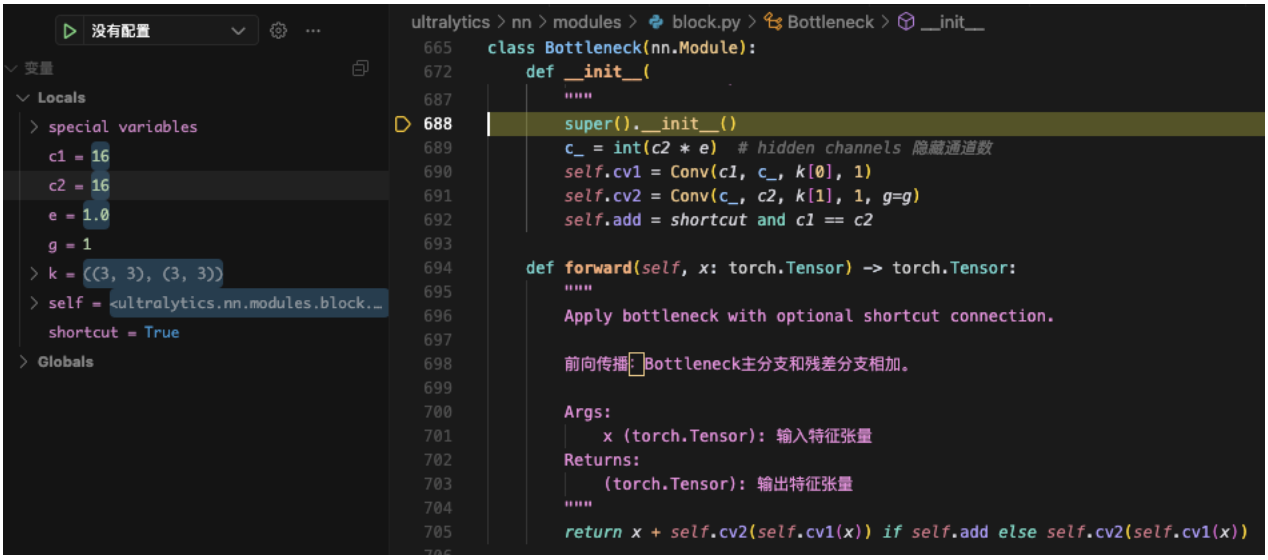
- C3k2



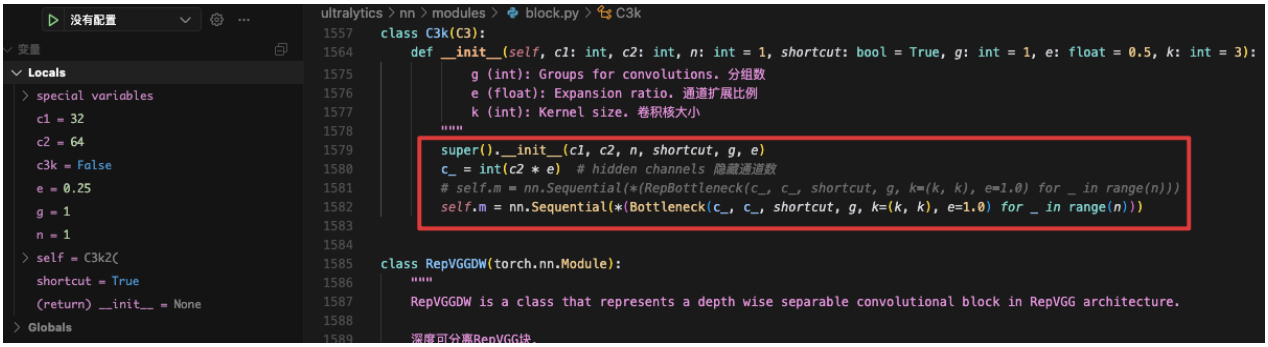
- C2f



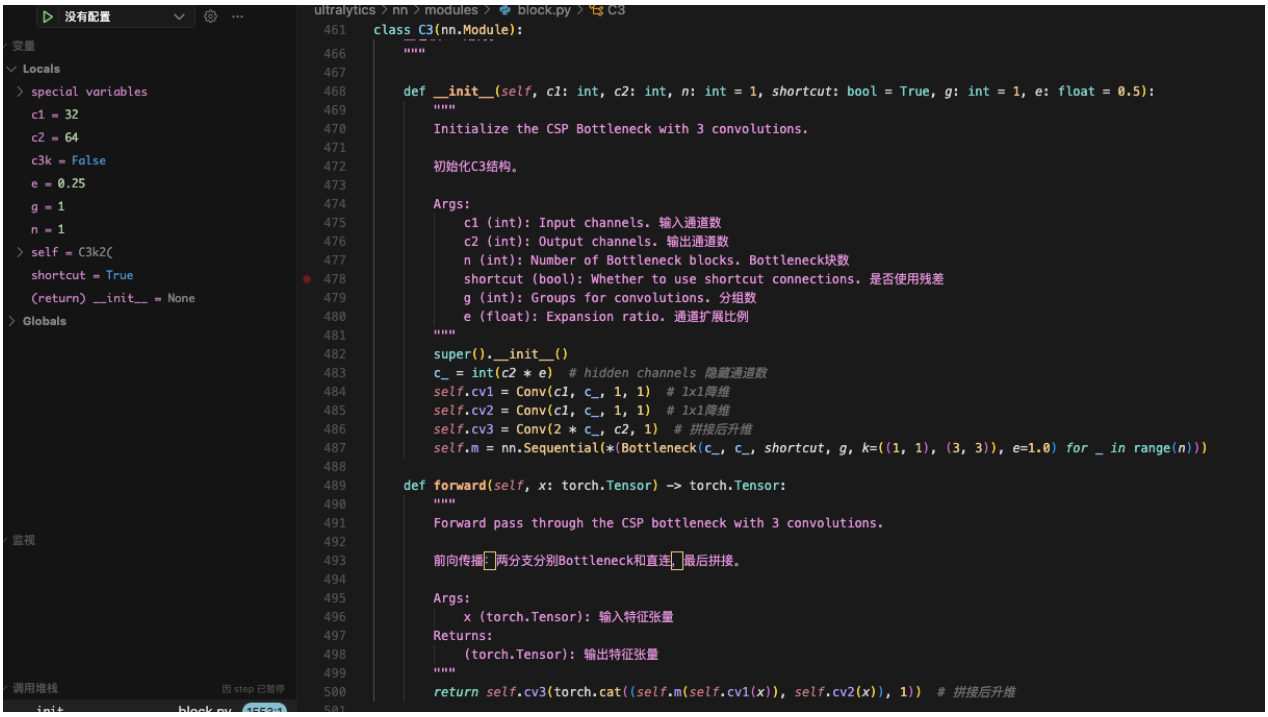
- Bottleneck



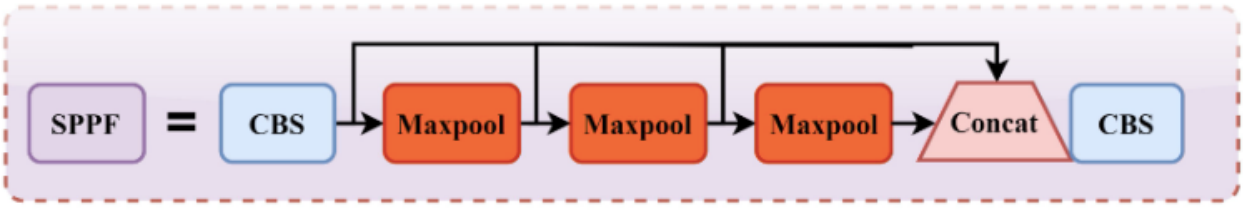
- C3k

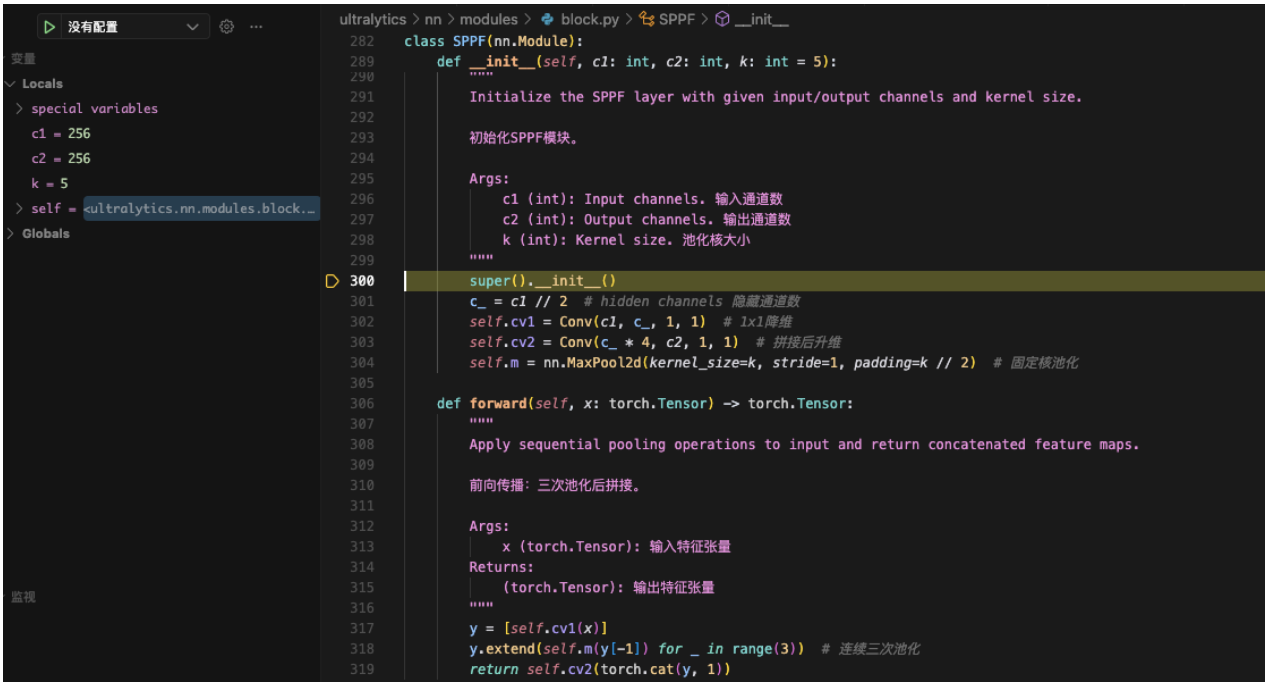

没有配置
变量
Locals
special variables
c1 = 32
c2 = 64
c3k = False
e = 0.25
g = 1
n = 1
self = C3k2(
shortcut = True
(return) __init__ = None
Globals
ultralytics > nn > modules > block.py > C3k
class C3k(C3):
def __init__(self, c1: int, c2: int, n: int = 1, shortcut: bool = True, g: int = 1, e: float = 0.5, k: int = 3):
g (int): Groups for convolutions. 分组数
e (float): Expansion ratio. 通道扩展比例
k (int): Kernel size. 卷积核大小
super().__init__(c1, c2, n, shortcut, g, e)
c_ = int(c2 * e) # hidden channels 隐藏通道数
# self.m = nn.Sequential(*(RepBottleneck(c_, c_, shortcut, g, k=(k, k), e=1.0) for _ in range(n)))
self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, k=(k, k), e=1.0) for _ in range(n)))
class RepVGGDW(torch.nn.Module):
RepVGGDW is a class that represents a depth wise separable convolutional block in RepVGG architecture.
深度可分离RepVGG块

- C3


没有配置
变量
Locals
special variables
c1 = 32
c2 = 64
c3k = False
e = 0.25
g = 1
n = 1
self = C3k2(
shortcut = True
(return) __init__ = None
Globals
监视
调用堆栈
ultralytics > nn > modules > block.py > C3
class C3(nn.Module):
Initialize the CSP Bottleneck with 3 convolutions.
初始化C3结构。
Args:
c1 (int): Input channels. 输入通道数
c2 (int): Output channels. 输出通道数
n (int): Number of Bottleneck blocks. Bottleneck块数
shortcut (bool): Whether to use shortcut connections. 是否使用残差
g (int): Groups for convolutions. 分组数
e (float): Expansion ratio. 通道扩展比例
super().__init__()
c_ = int(c2 * e) # hidden channels 隐藏通道数
self.cv1 = Conv(c1, c_, 1, 1) # 1x1降维
self.cv2 = Conv(c1, c_, 1, 1) # 1x1降维
self.cv3 = Conv(2 * c_, c2, 1) # 拼接后升维
self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, k=((1, 1), (3, 3)), e=1.0) for _ in range(n)))
def forward(self, x: torch.Tensor) -> torch.Tensor:
Forward pass through the CSP bottleneck with 3 convolutions.
前向传播：两分支分别Bottleneck和直连，最后拼接。
Args:
x (torch.Tensor): 输入特征张量
Returns:
(torch.Tensor): 输出特征张量
return self.cv3(torch.cat((self.m(self.cv1(x)), self.cv2(x)), 1)) # 拼接后升维

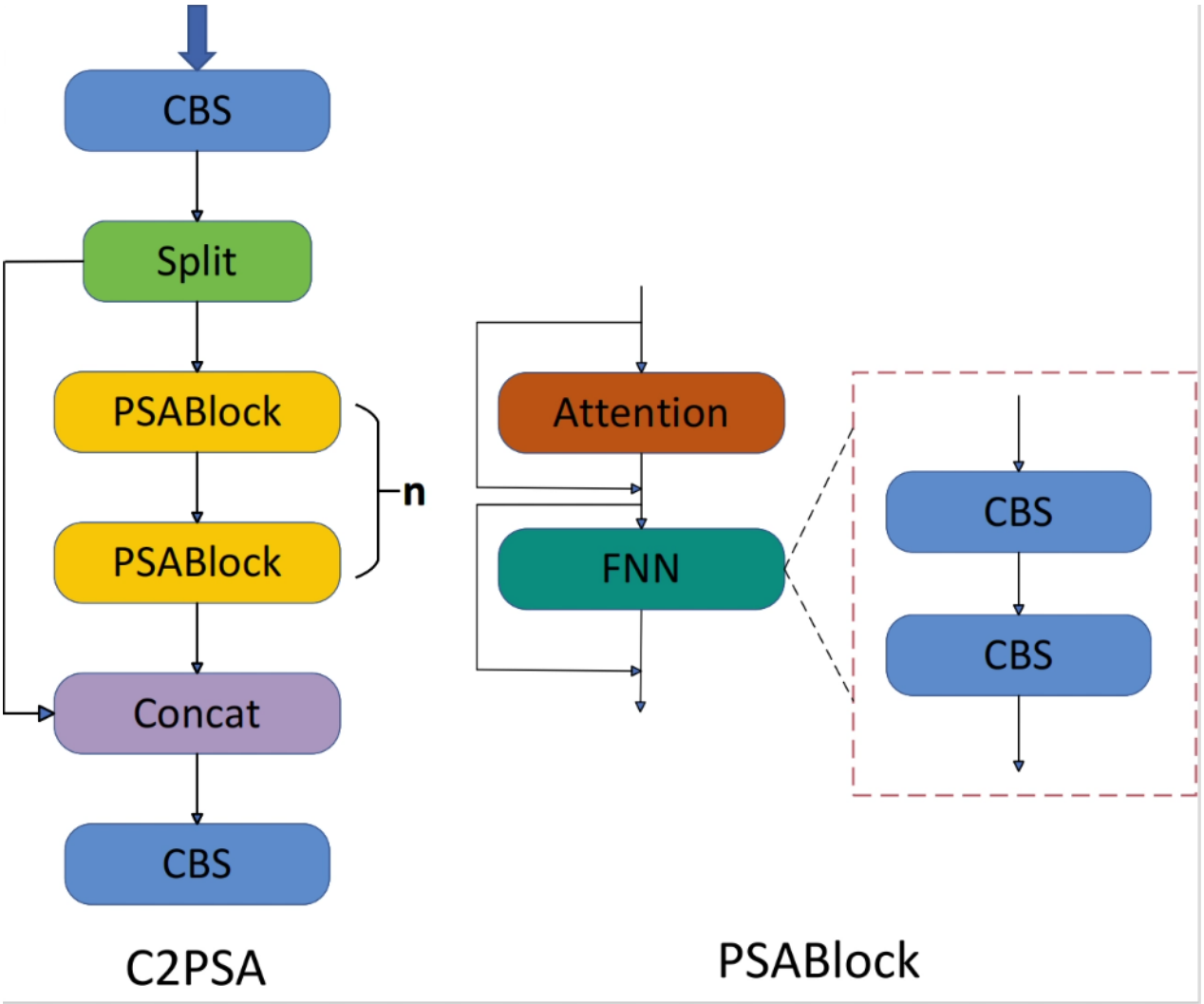
3.4.3 SPPF


SPPF
=
CBS
Maxpool
Maxpool
Maxpool
Concat
CBS

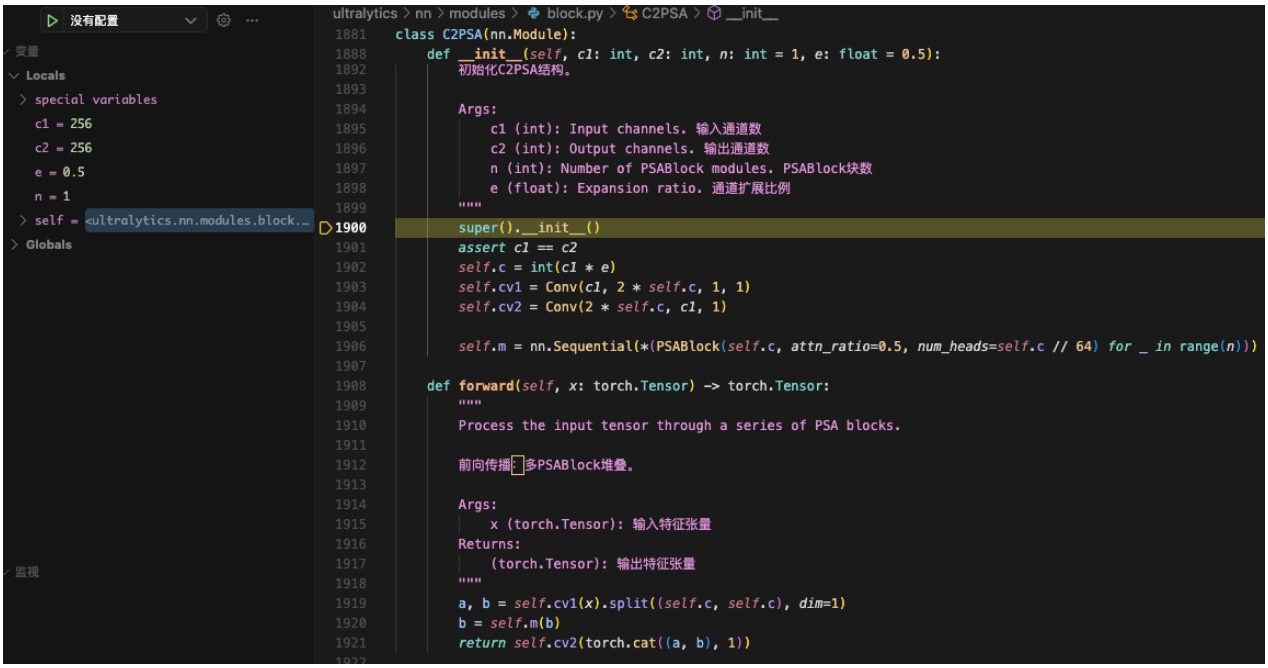


3.4.4 C2PSA

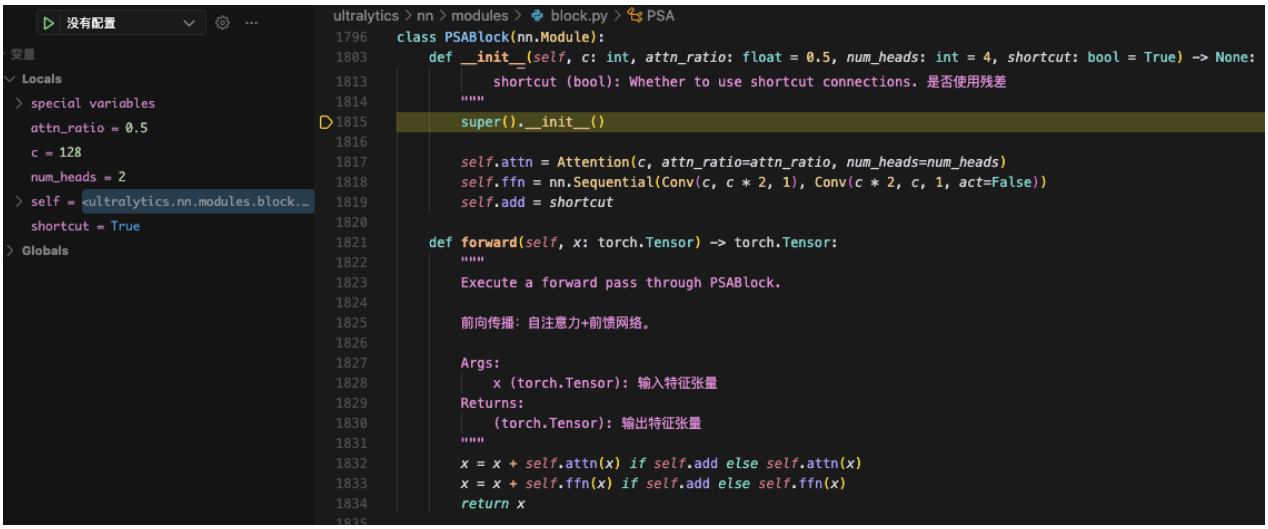
C2PSA是对 `C2f` 模块的扩展，它结合了PSA(Pointwise Spatial Attention)块，用于增强特征提取和注意力机制。通过在标准 `C2f` 模块中引入 PSA 块，C2PSA实现了更强大的注意力机制，从而提高了模型对重要特征的捕捉能力。



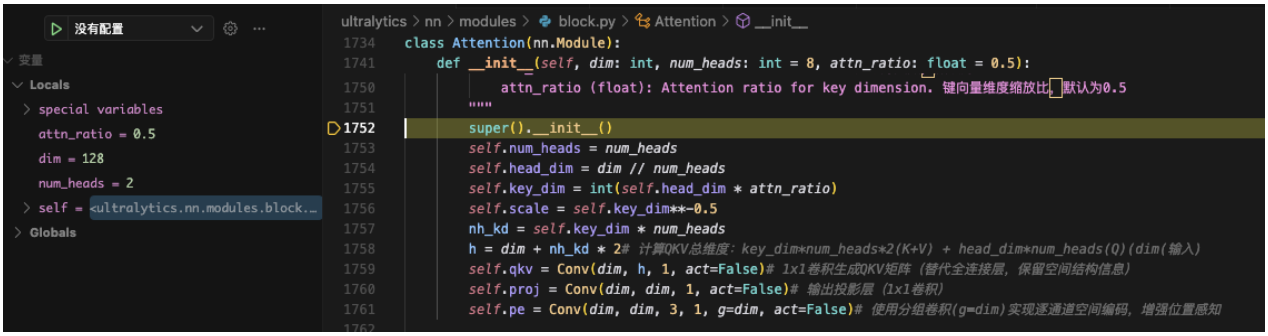
- C2PSA

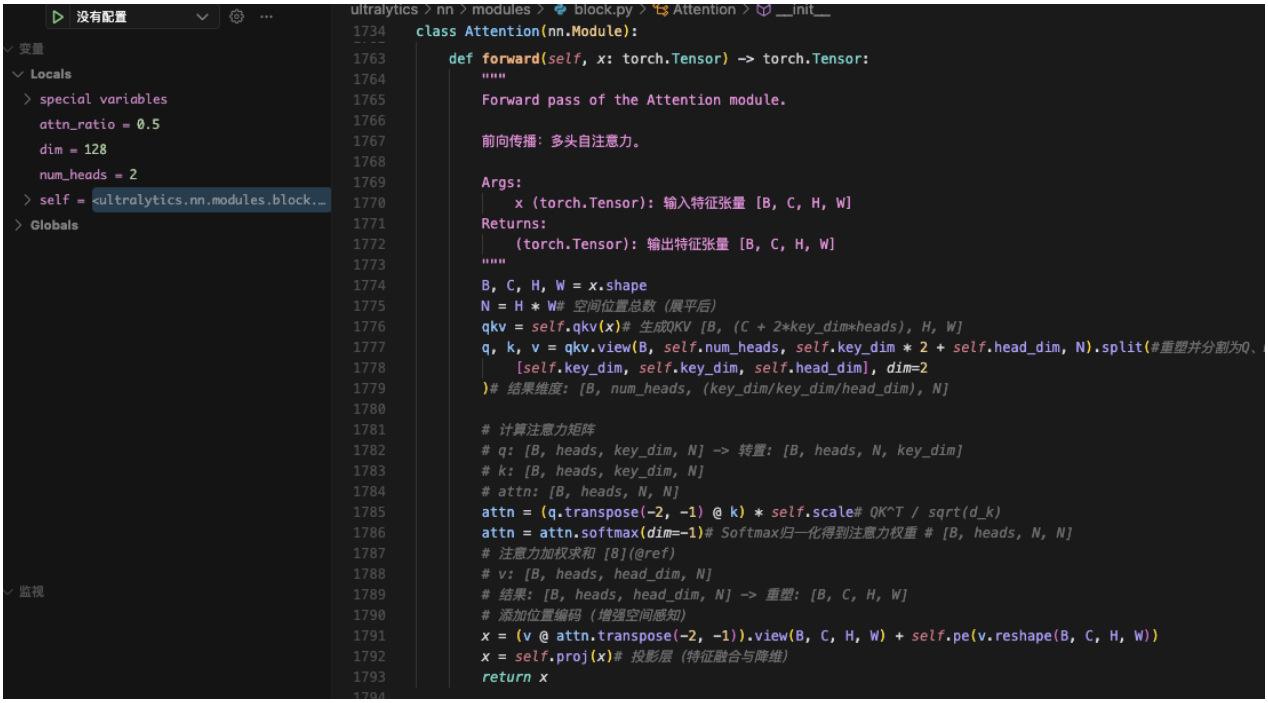


- PSABlock



- Attention





3.4.5 Neck

parse_model函数执行过程中会输出模型结构信息：

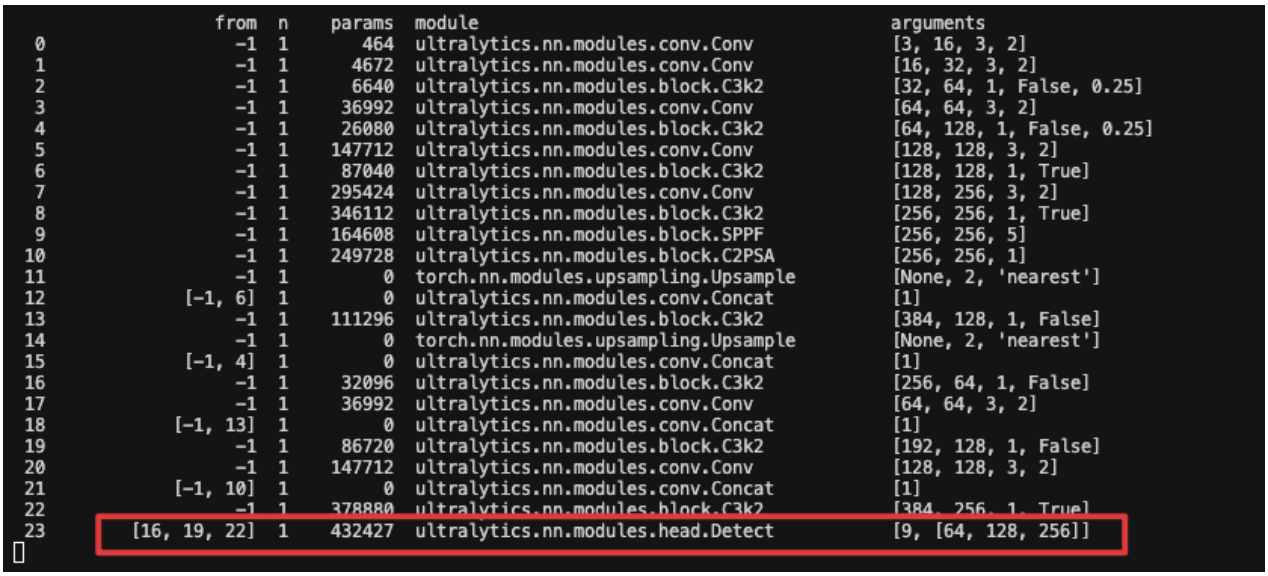
```
                 from  n    params  module                                       arguments
  0                -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                -1  1      6640  ultralytics.nn.modules.block.C3k2            [32, 64, 1, False, 0.25]
  3                -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
  4                -1  1     26080  ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
  5                -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
  6                -1  1     87040  ultralytics.nn.modules.block.C3k2            [128, 128, 1, True]
  7                -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                -1  1    346112  ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
  9                -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                -1  1    249728  ultralytics.nn.modules.block.C2PSA           [256, 256, 1]
 11                -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 12            [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 13                -1  1    111296  ultralytics.nn.modules.block.C3k2            [384, 128, 1, False]
 14                -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 15            [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 16                -1  1     32096  ultralytics.nn.modules.block.C3k2            [256, 64, 1, False]
 17                -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 18           [-1, 13]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 19                -1  1     86720  ultralytics.nn.modules.block.C3k2            [192, 128, 1, False]
 20                -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 21           [-1, 10]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 22                -1  1    378880  ultralytics.nn.modules.block.C3k2            [384, 256, 1, True]
```

- from：这个参数代表从哪一层获得输入，-1就表示从上一层获得输入，[-1, 6]就表示从上一层和第6层这两层获得输入。第一层比较特殊，这里第一层上一层 没有输入，from默认-1就好了。
- number：这个参数表示模块重复的次数，如果为2则表示该模块重复3次，这里并不一定是这个模块的重复次数，也有可能是这个模块中的子模块重复的次数。对于C3k2模块来说，这个number就代表C3k2中Bottelneck或是C3k模块重复的次数。
- module：这个就代表你这层使用的模块的名称，比如你第一层使用了Conv模块，第三层使用了C3k2模块。
- args：表示这个模块需要传入的参数，第一个参数均表示该层的输出通道数。对于第一层conv参数【64,3，2】：64代表输出通道数，3代表卷积核大小k，2代表stride步长。每层输入通道数，默认是上一层的输出通道数。

3.4.6 Head

YOLO11在head部分的cls分支上使用深度可分离卷积，具体代码如下，cv2边界框回归分支，cv3分类分支。

`Detect`的`from`有三个数：`16`，`19`，`22`，这三个就是最终网络的输出特征图，分别对应`P3`，`P4`，`P5`。



3.5 前向传播和损失计算

3.5.1 加载权重

- model.load(*weights*)

调用`DetectionModel`的load方法

```
142         model = DetectionModel(cfg, nc=self.data["nc"], ch=self.data["channels"], verbose=verbose and RANK == -1)
143         if weights:
144             model.load(weights)
145         return model
```

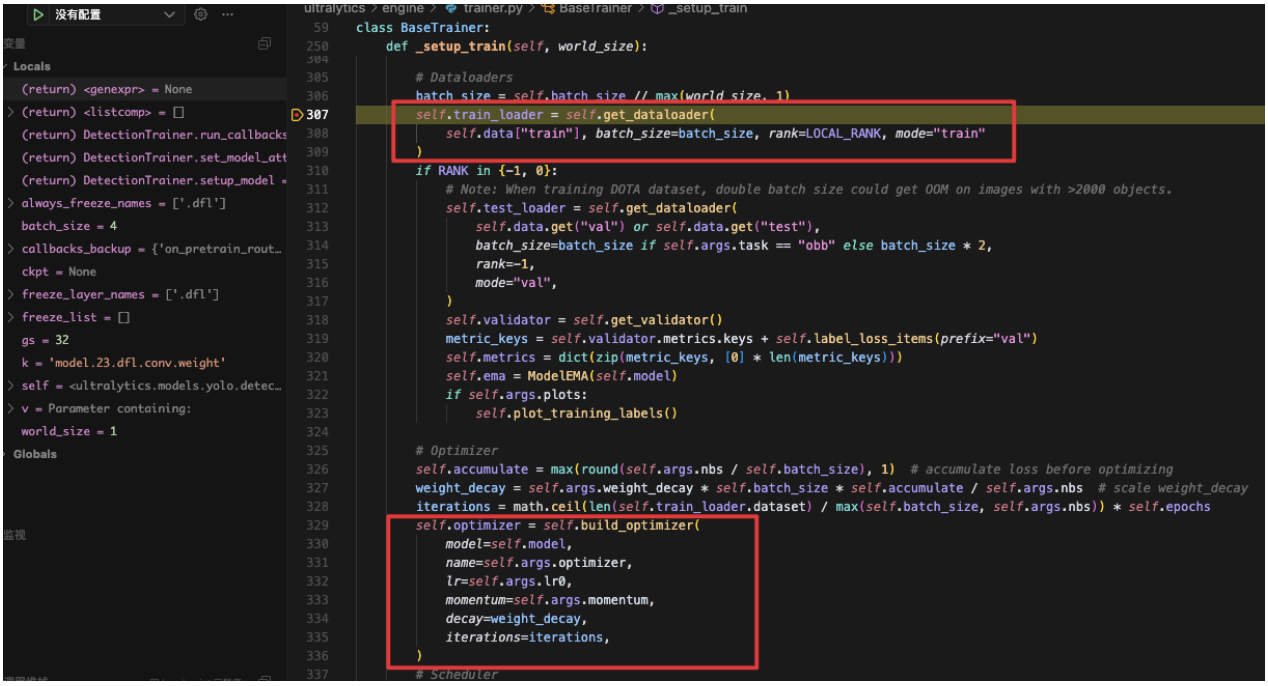
最终执行的是基类`BaseModel`的laod方法：

```
97  class BaseModel(torch.nn.Module):
282     def _apply(self, fn):
299             m.strides = fn(m.strides)
300         return self
301
302     def load(self, weights, verbose=True):
303         """
304         Load weights into the model.
305
306         Args:
307             weights (dict | torch.nn.Module): The pre-trained weights to be loaded.
308             verbose (bool, optional): Whether to log the transfer progress.
309         """
310         model = weights["model"] if isinstance(weights, dict) else weights  # torchvision models are not dicts
311         csd = model.float().state_dict()  # checkpoint state_dict as FP32
312         updated_csd = intersect_dicts(csd, self.state_dict())  # intersect
313         self.load_state_dict(updated_csd, strict=False)  # load
314         len_updated_csd = len(updated_csd)
315         first_conv = "model.0.conv.weight"  # hard-coded to yolo models for now
316         # mostly used to boost multi-channel training
317         state_dict = self.state_dict()
318         if first_conv not in updated_csd and first_conv in state_dict:
319             c1, c2, h, w = state_dict[first_conv].shape
320             cc1, cc2, ch, cw = csd[first_conv].shape
321             if ch == h and cw == w:
322                 c1, c2 = min(c1, cc1), min(c2, cc2)
323                 state_dict[first_conv][:c1, :c2] = csd[first_conv][:c1, :c2]
324                 len_updated_csd += 1
325         if verbose:
326             LOGGER.info(f"Transferred {len_updated_csd}/{len(self.model.state_dict())} items from pretrained weights")
```

然后调用torch.nnModule的load_state_dict方法。

3.5.2 构建dataloader和optimizer

调用链：DetectTrainer.train—>BaseTrainer.train—>BaseTrainer._do_train—>BaseTrainer._setup_train



ModelEMA

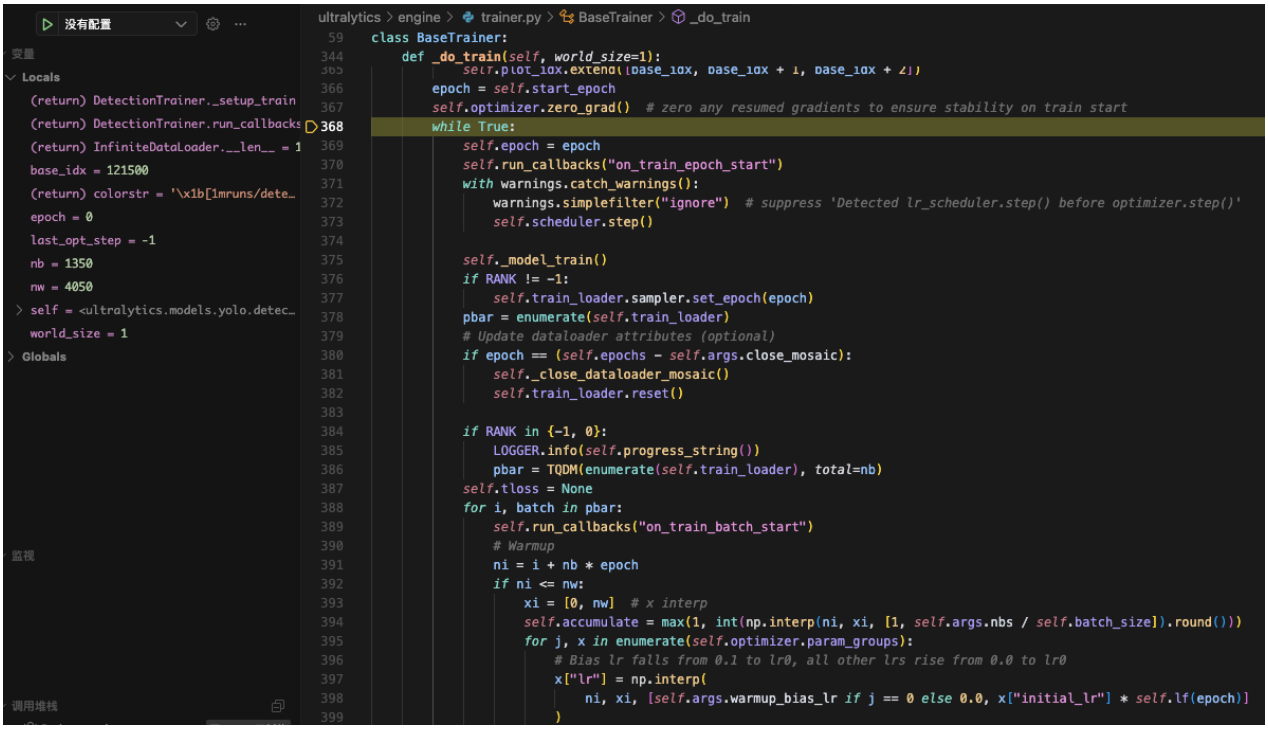
模型权重在最后的n步内，会在实际的最优点处抖动，所以我们取最后n步的平均，能使得模型更加鲁棒。ema是用来在test或val时才使用，用来更新参数。

$$mv_t = decay * mv_t - 1 + (1 - decay) * variable$$

```python
class ModelEMA:
    def __init__(self, model, decay=0.9999, tau=2000, updates=0):
        """
        self.ema = deepcopy(de_parallel(model)).eval()  # FP32 EMA
        self.updates = updates  # number of EMA updates
        self.decay = lambda x: decay * (1 - math.exp(-x / tau))  # decay exponential ramp (to help early epochs)
        for p in self.ema.parameters():
            p.requires_grad_(False)
        self.enabled = True

    def update(self, model):
        """
        Update EMA parameters.

        Args:
            model (nn.Module): Model to update EMA from.
        """
        if self.enabled:
            self.updates += 1
            d = self.decay(self.updates)

            msd = de_parallel(model).state_dict()  # model state_dict
            for k, v in self.ema.state_dict().items():
                if v.dtype.is_floating_point:  # true for FP16 and FP32
                    v *= d
                    v += (1 - d) * msd[k].detach()
                    # assert v.dtype == msd[k].dtype == torch.float32, f'{k}: EMA {v.dtype},  model {msd[k].dtype}'

```

3.5.3 训练主循环

调用链：DetectTrainer.train—>BaseTrainer.train—>BaseTrainer._do_train



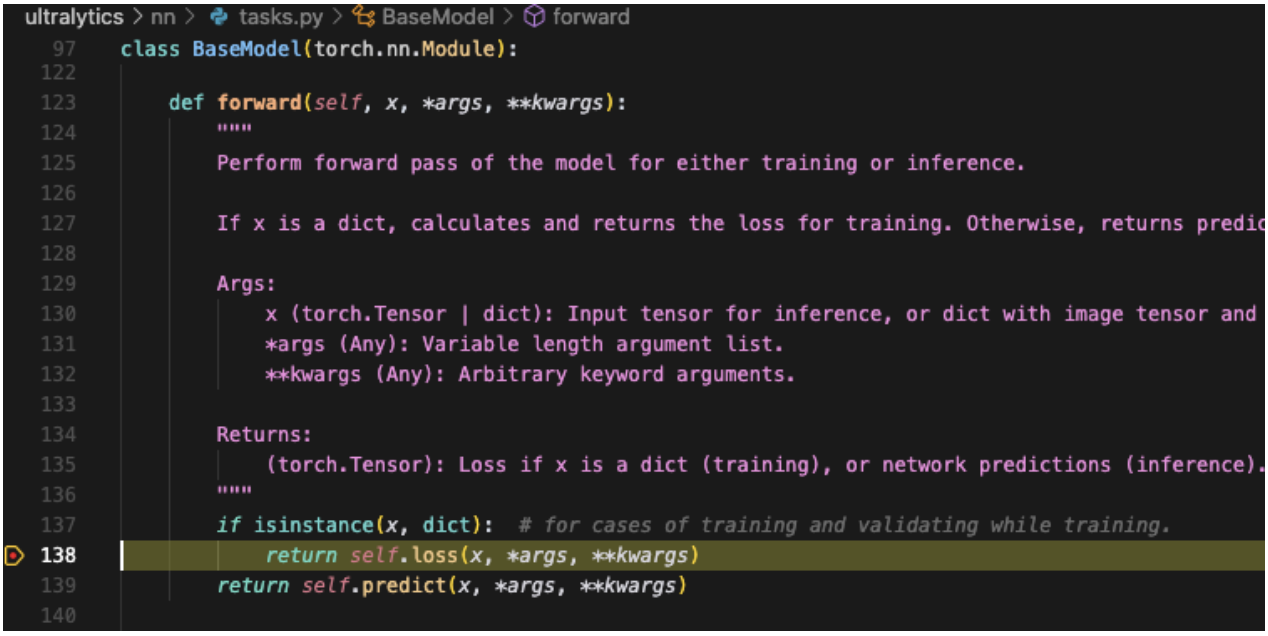
主循环中包含：**epoch**（warmup、Forward、Backward、Optimize、Log）、**Validation**、**Save model**、**Scheduler**、**Early Stopping**

```
344     def _do_train(self, world_size=1):
388         for i, batch in pbar:
389             self.run_callbacks("on_train_batch_start")
390             # Warmup
391             ni = i + nb * epoch
392             if ni <= nw:
393                 xi = [0, nw]  # x interp
394                 self.accumulate = max(1, int(np.interp(ni, xi, [1, self.args.nbs / self.batch_size]).round()))
395                 for j, x in enumerate(self.optimizer.param_groups):
396                     # Bias lr falls from 0.1 to lr0, all other lrs rise from 0.0 to lr0
397                     x["lr"] = np.interp(
398                         ni, xi, [self.args.warmup_bias_lr if j == 0 else 0.0, x["initial_lr"] * self.lf(epoch)]
399                     )
400                     if "momentum" in x:
401                         x["momentum"] = np.interp(ni, xi, [self.args.warmup_momentum, self.args.momentum])
402
403             # Forward
404             with autocast(self.amp):
405                 batch = self.preprocess_batch(batch)
406                 loss, self.loss_items = self.model(batch)#完成模型前向计算和损失计算
407                 self.loss = loss.sum()
408                 if RANK != -1:
409                     self.loss *= world_size
410                 self.tloss = (
411                     (self.tloss * i + self.loss_items) / (i + 1) if self.tloss is not None else self.loss_items
412                 )
413
414             # Backward
415             self.scaler.scale(self.loss).backward()
```

- preprocess_batch 准备图像数据

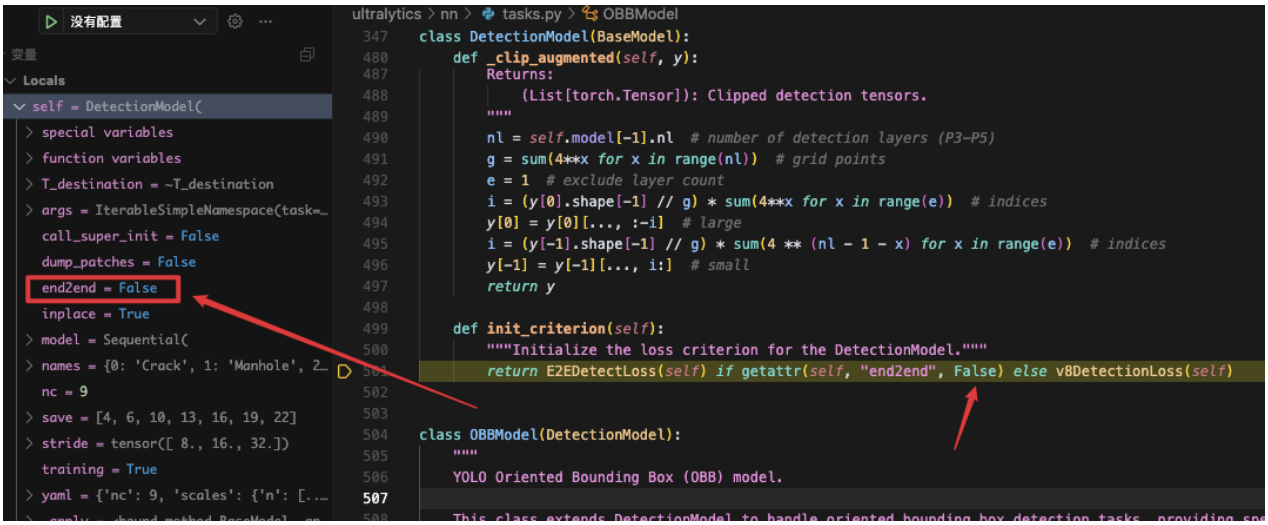
```
21  class DetectionTrainer(BaseTrainer):
92      def preprocess_batch(self, batch: Dict) -> Dict:
94          Preprocess a batch of images by scaling and converting to float.
95
96          Args:
97              batch (Dict): Dictionary containing batch data with 'img' tensor.
98
99          Returns:
100             (Dict): Preprocessed batch with normalized images.
101         """
102         batch["img"] = batch["img"].to(self.device, non_blocking=True).float() / 255
103         if self.args.multi_scale:
104             imgs = batch["img"]
105             sz = (
106                 random.randrange(int(self.args.imgsz * 0.5), int(self.args.imgsz * 1.5 + self.stride))
107                 // self.stride
108                 * self.stride
109             )  # size
110             sf = sz / max(imgs.shape[2:])  # scale factor
111             if sf != 1:
112                 ns = [
113                     math.ceil(x * sf / self.stride) * self.stride for x in imgs.shape[2:]
114                 ]  # new shape (stretched to gs-multiple)
115                 imgs = nn.functional.interpolate(imgs, size=ns, mode="bilinear", align_corners=False)
116             batch["img"] = imgs
117         return batch
```

- model(batch) 完成模型前向计算和损失计算

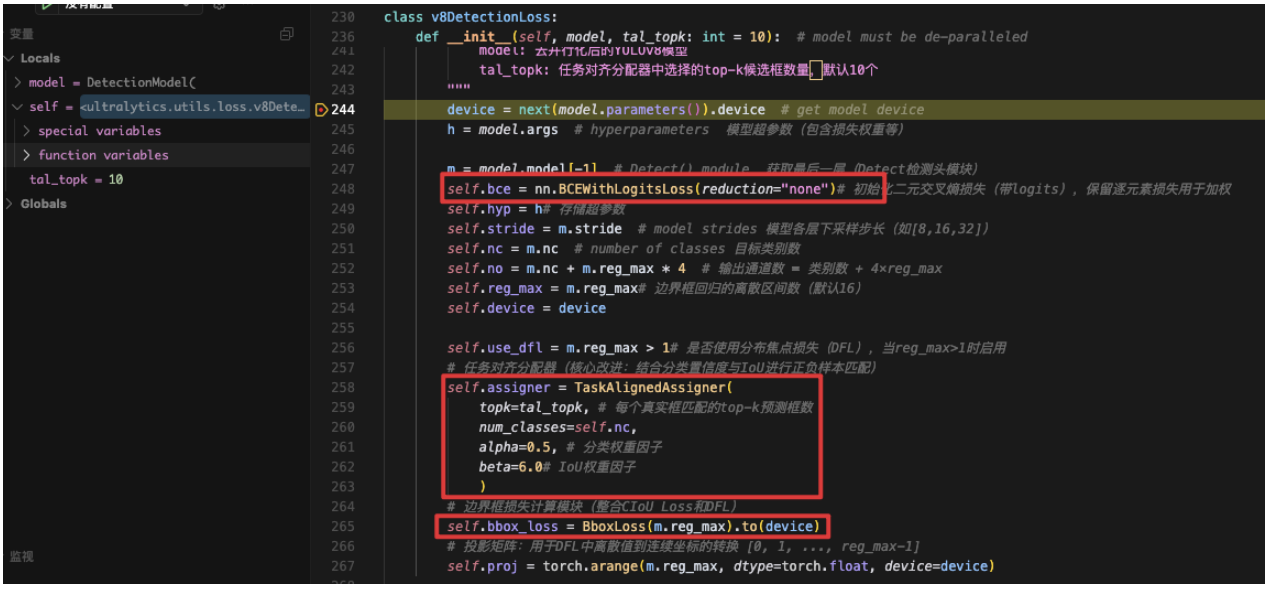


3.5.4 v8DetectionLoss实例化

执行*self*.init_criterion()，进入`DetectionModel`的`init_criterion`方法，其中实例化了`v8DetectionLoss`.



进入`v8DetectionLoss`可以看到损失由三部分组成，分别是：位置预测损失：DFL + CIoU Loss，类别预测损失：BCE Loss，此外使用了Task-Aligned Assigner匹配方式。



3.5.5 Forward

- 实例化v8DetectionLoss损失函数后，代码准备进行真正的Forward前向计算。

```
328     def loss(self, batch, preds=None):
329         """
330         Compute loss.
331 
332         Args:
333             batch (dict): Batch to compute loss on.
334             preds (torch.Tensor | List[torch.Tensor], optional): Predictions.
335         """
336         if getattr(self, "criterion", None) is None:
337             self.criterion = self.init_criterion() #初始化损失函数
338 
339         preds = self.forward(batch["img"]) if preds is None else preds #前向传播
340         return self.criterion(preds, batch)#损失计算
341 
```

- 依然调用BaseModel的forward方法，这次走到predict方法调用。

```
97  class BaseModel(torch.nn.Module):
123     def forward(self, x, *args, **kwargs):
134         Returns:
135             (torch.Tensor): Loss if x is a dict (training), or network predictions (inference).
136         """
137         if isinstance(x, dict):  # for cases of training and validating while training.
138             return self.loss(x, *args, **kwargs)
139         return self.predict(x, *args, **kwargs)
140 
```

- 由于没有打开数据增强，走到_predict_once方法。

```
97  class BaseModel(torch.nn.Module):
141     def predict(self, x, profile=False, visualize=False, augment=False, embed=None):
155         if augment:#是否使用数据增强
156             return self._predict_augment(x)
157         return self._predict_once(x, profile, visualize, embed)
158 
```

- 在_predict_once中，循环遍历model的各个子模块，进行前向计算，此时会执行各个子模块的forward方法。可以设置断点观察每一模块输出的特征图形状，对比架构图验证。

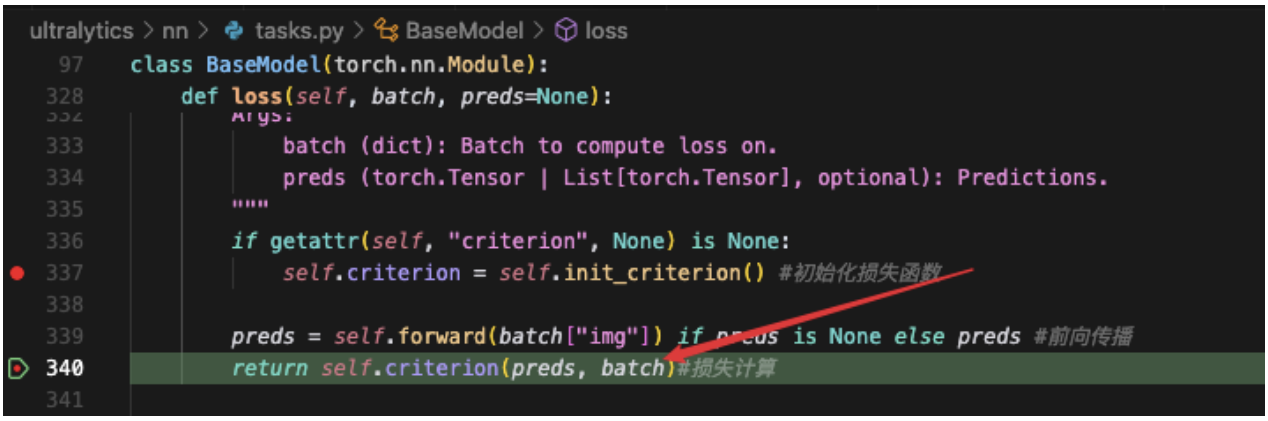
```python
class BaseModel(torch.nn.Module):
    def _predict_once(self, x, profile=False, visualize=False, embed=None):
        y, dt, embeddings = [], [], []  # outputs
        embed = frozenset(embed) if embed is not None else {-1}
        max_idx = max(embed)
        for m in self.model:
            if m.f != -1:  # if not from previous layer
                x = y[m.f] if isinstance(m.f, int) else [x if j == -1 else y[j] for j in m.f]  # from earlier layers
            if profile:
                self._profile_one_layer(m, x, dt)
            x = m(x)  # run
            y.append(x if m.i in self.save else None)  # save output
            if visualize:
                feature_visualization(x, m.type, m.i, save_dir=visualize)
            if m.i in embed:
                embeddings.append(torch.nn.functional.adaptive_avg_pool2d(x, (1, 1)).squeeze(-1).squeeze(-1))  # flatten
                if m.i == max_idx:
                    return torch.unbind(torch.cat(embeddings, 1), dim=0)
        return x
```

观察输出x：[4,73,80,80]、[4,73,40,40]、[4,73,20,20]

73=4x16+9

3.5.6 损失计算

Forward结束之后，进入loss计算环节：



进入`v8DetectionLoss`的`__call__`方法：

```
230  class v8DetectionLoss:
310      def __call__(self, preds: Any, batch: Dict[str, torch.Tensor]) -> Tuple[torch.Tensor, torch.Tensor]:
312          Calculate the sum of the loss for box, cls and dfl multiplied by batch size.
313          损失计算入口：整合边界框损失、分类损失、DFL损失
314          Args:
315              preds: 模型输出（多尺度特征图）
316              batch: 批次数据 {images, batch_idx, cls, bboxes}
317          Returns:
318              total_loss: 加权总损失
319              loss_components: 各损失分量[box, cls, dfl]
320          """
321          # 初始化损失分量：边界框损失/分类损失/DFL损失
322          loss = torch.zeros(3, device=self.device)  # box, cls, dfl
323          # 处理多尺度输出：拼接特征图 [batch, no, total_anchors]
324          feats = preds[1] if isinstance(preds, tuple) else preds
325          pred_distri, pred_scores = torch.cat([xi.view(feats[0].shape[0], self.no, -1) for xi in feats], 2).split(
326              (self.reg_max * 4, self.nc), 1
327          )
328          # 调整维度：[batch, anchors, channels]
329          pred_scores = pred_scores.permute(0, 2, 1).contiguous()
330          pred_distri = pred_distri.permute(0, 2, 1).contiguous()
331
332          dtype = pred_scores.dtype
333          batch_size = pred_scores.shape[0]
334          imgsz = torch.tensor(feats[0].shape[2:], device=self.device, dtype=dtype) * self.stride[0]  # image size (h,w)
335          # 生成锚点 & 计算特征图尺度
336          anchor_points, stride_tensor = make_anchors(feats, self.stride, 0.5)
337
338          # Targets  目标预处理
339          targets = torch.cat((batch["batch_idx"].view(-1, 1), batch["cls"].view(-1, 1), batch["bboxes"]), 1)
340          targets = self.preprocess(targets.to(self.device), batch_size, scale_tensor=imgsz[[1, 0, 1, 0]])
341          gt_labels, gt_bboxes = targets.split((1, 4), 2)  # cls, xyxy
342          mask_gt = gt_bboxes.sum(2, keepdim=True).gt_(0.0)
343
344          # Pboxes  解码预测框
```

该方法中主要进行以下操作：

3.5.6.1 预测值切分转置

将`preds[4,73,xx,xx]`切分为`bbox[4,64,8400]`距离分布分值和`cls[4,9,8400]`分值,然后转置成`bbox[4,8400,64]`和`cls[4,8400,9]`

```
323          # 处理多尺度输出：拼接特征图 [batch, no, total_anchors]
324          feats = preds[1] if isinstance(preds, tuple) else preds
325          pred_distri, pred_scores = torch.cat([xi.view(feats[0].shape[0], self.no, -1) for xi in feats], 2).split(
326              (self.reg_max * 4, self.nc), 1
327          )
328          # 调整维度：[batch, anchors, channels]
329          pred_scores = pred_scores.permute(0, 2, 1).contiguous()
330          pred_distri = pred_distri.permute(0, 2, 1).contiguous()
```

3.5.6.2 生成anchors坐标中心点

根据输入尺寸`[640,640]`和下采样步长`[8,16,32]`生成每层特征图的anchors中心点坐标

```
332          dtype = pred_scores.dtype
333          batch_size = pred_scores.shape[0]
334          imgsz = torch.tensor(feats[0].shape[2:], device=self.device, dtype=dtype) * self.stride[0]  # image size (h,w)
335          # 生成锚点 & 计算特征图尺度
336          anchor_points, stride_tensor = make_anchors(feats, self.stride, 0.5)
```

3.5.6.3 真是标签预处理

对输入图像的真实目标进行处理，处理成`[640,640]`规格下的[class, x1, y1, x2, y2]形式

```
338          # Targets  目标预处理
339          targets = torch.cat((batch["batch_idx"].view(-1, 1), batch["cls"].view(-1, 1), batch["bboxes"]), 1)
340          targets = self.preprocess(targets.to(self.device), batch_size, scale_tensor=imgsz[[1, 0, 1, 0]])
341          gt_labels, gt_bboxes = targets.split((1, 4), 2)  # cls, xyxy
342          mask_gt = gt_bboxes.sum(2, keepdim=True).gt_(0.0)
```

```python
    def preprocess(self, targets: torch.Tensor, batch_size: int, scale_tensor: torch.Tensor) -> torch.Tensor:
        """预处理真实标签：转换格式并缩放坐标到特征图尺度
        Args:
            targets: 原始标签 [image_idx, class, x, y, w, h]
            batch_size: 当前批次大小
            scale_tensor: 特征图相对于原图的缩放因子 [s_w, s_h, s_w, s_h]
        Returns:
            处理后的标签 [class, x1, y1, x2, y2]（xyxy格式）
        """
        nl, ne = targets.shape
        if nl == 0:
            out = torch.zeros(batch_size, 0, ne - 1, device=self.device)
        else:
            i = targets[:, 0]  # image index   提取图像索引
            _, counts = i.unique(return_counts=True)# 统计每张图的目标数
            counts = counts.to(dtype=torch.int32)
            out = torch.zeros(batch_size, counts.max(), ne - 1, device=self.device)
            for j in range(batch_size):# 按图像索引填充数据
                matches = i == j
                if n := matches.sum():
                    # 保留[class, x, y, w, h] 并转换xywh→xyxy
                    out[j, :n] = targets[matches, 1:]
            out[..., 1:5] = xywh2xyxy(out[..., 1:5].mul_(scale_tensor))# 坐标缩放：适配当前特征图尺度
        return out
```

3.5.6.4 解码预测框

这里可以看到使用了DFL，

1. 离散概率分布建模

DFL将连续坐标值建模为**离散区间上的概率分布**。假设坐标真实值为 y，将其离散化为 n 个点 $y_0, y_1, \ldots, y_{n-1}$，模型预测每个点的概率 $P(y_i)$。这里的`n=16`

2. 积分计算连续值

通过加权求和（积分）得到最终预测坐标：

$$\hat{y} = \sum_{i=0}^{n-1} P(y_i) \cdot y_i$$

```python
class v8DetectionLoss:

    def bbox_decode(self, anchor_points: torch.Tensor, pred_dist: torch.Tensor) -> torch.Tensor:
        """从预测分布解码边界框坐标（DFL核心操作）
        Args:
            anchor_points: 锚点坐标 [num_anchors, 2]
            pred_dist: 预测分布 [batch, num_anchors, 4*reg_max]
        Returns:
            解码后的边界框 [x1, y1, x2, y2]（xyxy格式）
        """
        if self.use_dfl:
            b, a, c = pred_dist.shape  # batch, anchors, channels
            # 重塑→softmax→积分：将离散分布转为连续坐标
            pred_dist = pred_dist.view(b, a, 4, c // 4).softmax(3).matmul(self.proj.type(pred_dist.dtype))# ∑(P(
            # pred_dist = pred_dist.view(b, a, c // 4, 4).transpose(2,3).softmax(3).matmul(self.proj.type(pred_d
            # pred_dist = (pred_dist.view(b, a, c // 4, 4).softmax(2) * self.proj.type(pred_dist.dtype).view(1,
        return dist2bbox(pred_dist, anchor_points, xywh=False)# 分布→边界框坐标（相对锚点的偏移）
```

3.5.6.5 `TaskAligned Assigner`任务对齐分配

任务对齐分配：匹配预测框与真实框

代码进入`TaskAligned Assigner`类的`_forward`方法：

```python
class TaskAlignedAssigner(nn.Module):
    def _forward(self, pd_scores, pd_bboxes, anc_points, gt_labels, gt_bboxes, mask_gt):
        """
        # 步骤1：获取正样本掩码、对齐度量和IoU
        mask_pos, align_metric, overlaps = self.get_pos_mask(
            pd_scores, pd_bboxes, gt_labels, gt_bboxes, anc_points, mask_gt
        )
        # 步骤2：选择最高重叠的预测框（解决多gt竞争问题）
        target_gt_idx, fg_mask, mask_pos = self.select_highest_overlaps(mask_pos, overlaps, self.n_max_boxes)

        # Assigned target
        # 步骤3：生成分配目标（标签/边界框/分数）
        target_labels, target_bboxes, target_scores = self.get_targets(gt_labels, gt_bboxes, target_gt_idx, fg_mask)

        # Normalize
        # 步骤4：归一化对齐度量（增强困难样本权重）
        align_metric *= mask_pos
        pos_align_metrics = align_metric.amax(dim=-1, keepdim=True)  # b, max_num_obj # 每个gt的最大对齐度量 [b, max_gt, 1]
        pos_overlaps = (overlaps * mask_pos).amax(dim=-1, keepdim=True)  # b, max_num_obj # 每个gt的最大IoU [b, max_gt, 1]
        # 归一化公式：align_metric * (IoU / max_align_metric)
        norm_align_metric = (align_metric * pos_overlaps / (pos_align_metrics + self.eps)).amax(-2).unsqueeze(-1)
        target_scores = target_scores * norm_align_metric

        return target_labels, target_bboxes, target_scores, fg_mask.bool(), target_gt_idx
```

该方法中主要4个步骤：

1.get_pos_mask:获取正样本掩码、对齐度量、并为每个真实框选择top-k匹配的锚点

```python
class TaskAlignedAssigner(nn.Module):
    def get_pos_mask(self, pd_scores, pd_bboxes, gt_labels, gt_bboxes, anc_points, mask_gt):
        """
        # 步骤1：筛选位于gt框内的锚点 [bs,max_num_obj,h*w]
        mask_in_gts = self.select_candidates_in_gts(anc_points, gt_bboxes)
        # Get anchor_align metric, (b, max_num_obj, h*w)
        # 步骤2：计算对齐度量和IoU
        align_metric, overlaps = self.get_box_metrics(pd_scores, pd_bboxes, gt_labels, gt_bboxes, mask_in_gts * mask_gt)
        # Get topk_metric mask, (b, max_num_obj, h*w)
        # 步骤3：选择每个gt的top-k锚点
        mask_topk = self.select_topk_candidates(align_metric, topk_mask=mask_gt.expand(-1, -1, self.topk).bool())
        # Merge all mask to a final mask, (b, max_num_obj, h*w)
        # 综合三种条件生成最终掩码
        mask_pos = mask_topk * mask_in_gts * mask_gt

        return mask_pos, align_metric, overlaps
```

首先使用anchors的中心点坐标anc_points和真实框ltrb坐标，计算中心点落在了哪些gt中；

```python
class TaskAlignedAssigner(nn.Module):
    def select_candidates_in_gts(xy_centers, gt_bboxes, eps=1e-9):
        n_anchors = xy_centers.shape[0]
        bs, n_boxes, _ = gt_bboxes.shape
        #计算锚点到gt框边界的距离 [b, n_boxes, num_anchors, 4]
        lt, rb = gt_bboxes.view(-1, 1, 4).chunk(2, 2)  # left-top, right-bottom
        #锚点-真实框左上角，真实右下角-锚点
        bbox_deltas = torch.cat((xy_centers[None] - lt, rb - xy_centers[None]), dim=2).view(bs, n_boxes, n_anchors, -1)
        #检查所有方向的距离是否为正（min(dx,dy) > 0）  两个值都为正的就表明锚点在真实框内
        return bbox_deltas.amin(3).gt_(eps)
```

然后计算对齐度量：对齐度量公式 $align_metric = (class_score)^{\alpha} \times (IoU)^{\beta}$

```python
class TaskAlignedAssigner(nn.Module):
    def get_box_metrics(self, pd_scores, pd_bboxes, gt_labels, gt_bboxes, mask_gt):
            overlaps (torch.Tensor): IoU overlaps between predicted and ground truth boxes.
        """
        na = pd_bboxes.shape[-2]# 锚点数量
        mask_gt = mask_gt.bool()  # b, max_num_obj, h*w  转换为布尔掩码

        # 初始化IoU和分类分数矩阵
        overlaps = torch.zeros([self.bs, self.n_max_boxes, na], dtype=pd_bboxes.dtype, device=pd_bboxes.device)
        bbox_scores = torch.zeros([self.bs, self.n_max_boxes, na], dtype=pd_scores.dtype, device=pd_scores.device)
        # 构建索引：获取每个gt标签对应的预测分数
        ind = torch.zeros([2, self.bs, self.n_max_boxes], dtype=torch.long)  # 2, b, max_num_obj
        ind[0] = torch.arange(end=self.bs).view(-1, 1).expand(-1, self.n_max_boxes)  # b, max_num_obj
        ind[1] = gt_labels.squeeze(-1)  # b, max_num_obj
        # Get the scores of each grid for each gt cls
        # 提取对应类别的预测分数
        bbox_scores[mask_gt] = pd_scores[ind[0], :, ind[1]][mask_gt]  # b, max_num_obj, h*w

        # (b, max_num_obj, 1, 4), (b, 1, h*w, 4) 计算IoU（仅对有效区域）
        pd_boxes = pd_bboxes.unsqueeze(1).expand(-1, self.n_max_boxes, -1, -1)[mask_gt]
        gt_boxes = gt_bboxes.unsqueeze(2).expand(-1, -1, na, -1)[mask_gt]
        overlaps[mask_gt] = self.iou_calculation(gt_boxes, pd_boxes)#计算Ciou
        # 任务对齐度量公式：align_metric=(class_score)^α × (IoU)^β
        align_metric = bbox_scores.pow(self.alpha) * overlaps.pow(self.beta)
        return align_metric, overlaps
```

最后，根据对齐度量选择每个gt的top-k锚点，这里使用了稀疏矩阵，因为对齐度量矩阵metrics极度稀疏，很多0值。

```python
class TaskAlignedAssigner(nn.Module):
    def select_topk_candidates(self, metrics, topk_mask=None):
        """
        # # 获取top-k索引(b, max_num_obj, topk)
        topk_metrics, topk_idxs = torch.topk(metrics, self.topk, dim=-1, largest=True)
        #生成top-k掩码（防止选择无效候选）
        if topk_mask is None:
            topk_mask = (topk_metrics.max(-1, keepdim=True)[0] > self.eps).expand_as(topk_idxs)
        # 构建稀疏矩阵标记top-k位置(b, max_num_obj, topk)
        topk_idxs.masked_fill_(~topk_mask, 0)

        # (b, max_num_obj, topk, h*w) -> (b, max_num_obj, h*w)
        count_tensor = torch.zeros(metrics.shape, dtype=torch.int8, device=topk_idxs.device)
        ones = torch.ones_like(topk_idxs[:, :, :1], dtype=torch.int8, device=topk_idxs.device)
        for k in range(self.topk):#通过scatter_add_高效标记top-k
            # Expand topk_idxs for each value of k and add 1 at the specified positions
            count_tensor.scatter_add_(-1, topk_idxs[:, :, k : k + 1], ones)
        # Filter invalid bboxes 过滤重复标记（确保每个位置只被计数一次）
        count_tensor.masked_fill_(count_tensor > 1, 0)

        return count_tensor.to(metrics.dtype)
```

2.select_highest_overlaps:选择最高重叠的预测框（解决多gt竞争问题）

```python
class TaskAlignedAssigner(nn.Module):
    def select_highest_overlaps(mask_pos, overlaps, n_max_boxes):
        """
        # Convert (b, n_max_boxes, h*w) -> (b, h*w)
        fg_mask = mask_pos.sum(-2)
        #处理多gt匹配的锚点（选择IoU最高的gt）
        if fg_mask.max() > 1:  # one anchor is assigned to multiple gt_bboxes
            mask_multi_gts = (fg_mask.unsqueeze(1) > 1).expand(-1, n_max_boxes, -1)  # (b, n_max_boxes, h*w)
            max_overlaps_idx = overlaps.argmax(1)  # (b, h*w)
            # 构建最高IoU的one-hot编码
            is_max_overlaps = torch.zeros(mask_pos.shape, dtype=mask_pos.dtype, device=mask_pos.device)
            is_max_overlaps.scatter_(1, max_overlaps_idx.unsqueeze(1), 1)
            # 更新掩码：仅保留最高IoU的gt
            mask_pos = torch.where(mask_multi_gts, is_max_overlaps, mask_pos).float()  # (b, n_max_boxes, h*w)
            fg_mask = mask_pos.sum(-2)
        # Find each grid serve which gt(index)
        target_gt_idx = mask_pos.argmax(-2)  # (b, h*w)
        return target_gt_idx, fg_mask, mask_pos
```

3.get_targets:生成分配目标（标签/边界框/分数））

```
class TaskAlignedAssigner(nn.Module):
    def get_targets(self, gt_labels, gt_bboxes, target_gt_idx, fg_mask):
        # 转换gt索引为全局索引Assigned target labels, (b, 1)
        batch_ind = torch.arange(end=self.bs, dtype=torch.int64, device=gt_labels.device)[..., None]
        target_gt_idx = target_gt_idx + batch_ind * self.n_max_boxes  # (b, h*w)
        #提取分配的标签和边界框
        target_labels = gt_labels.long().flatten()[target_gt_idx]  # (b, h*w)

        # Assigned target boxes, (b, max_num_obj, 4) -> (b, h*w, 4)
        target_bboxes = gt_bboxes.view(-1, gt_bboxes.shape[-1])[target_gt_idx]

        # Assigned target scores
        target_labels.clamp_(0)

        #生成目标分数矩阵（高效替代one_hot） 10x faster than F.one_hot()
        target_scores = torch.zeros(
            (target_labels.shape[0], target_labels.shape[1], self.num_classes),
            dtype=torch.int64,
            device=target_labels.device,
        )  # (b, h*w, 80)
        target_scores.scatter_(2, target_labels.unsqueeze(-1), 1)# 在类别维度填充1
        #应用前景掩码（背景位置置零）
        fg_scores_mask = fg_mask[:, :, None].repeat(1, 1, self.num_classes)  # (b, h*w, 80)
        target_scores = torch.where(fg_scores_mask > 0, target_scores, 0)

        return target_labels, target_bboxes, target_scores
```

4.归一化对齐度量（增强困难样本权重）

归一化公式：align_metric * IoU / max_align_metric

3.5.6.6 分类损失计算

类损失：BCE二元交叉熵（仅正样本）

```
        # Cls loss
        # loss[1] = self.varifocal_loss(pred_scores, target_scores, target_labels) / target_scores_sum  # VFL way
        # 1. 分类损失：二元交叉熵（仅正样本）
        loss[1] = self.bce(pred_scores, target_scores.to(dtype)).sum() / target_scores_sum  # BCE
```

3.5.6.7 Bbox loss计算

边界框损失：含CIoU Loss + DFL（仅正样本）

```python
class BboxLoss(nn.Module):
    def forward(
        """
        # ================== IoU损失计算 ==================
        # 计算样本权重：基于分类置信度加权（困难样本权重更高）
        # target_scores.sum(-1): 每个锚点的类别分数总和 [bs, num_anchors]
        # [fg_mask]: 仅选取前景样本 [num_fg_samples]
        weight = target_scores.sum(-1)[fg_mask].unsqueeze(-1)
        # 计算CIoU (Complete IoU) [1,4](@ref)
        # 仅在前景样本上计算，减少计算量
        iou = bbox_iou(pred_bboxes[fg_mask], target_bboxes[fg_mask], xywh=False, CIoU=True)
        # IoU损失公式: (1 - IoU) * 权重
        loss_iou = ((1.0 - iou) * weight).sum() / target_scores_sum

        # DFL loss
        # ================== DFL损失计算 ==================
        if self.dfl_loss:
            # 将真实框转换为距离表示（ltrb格式）
            target_ltrb = bbox2dist(anchor_points, target_bboxes, self.dfl_loss.reg_max - 1)
            # 重塑预测分布: [num_fg_samples, reg_max] 提取真实距离: [num_fg_samples, 4]  计算损失并加权
            loss_dfl = self.dfl_loss(pred_dist[fg_mask].view(-1, self.dfl_loss.reg_max), target_ltrb[fg_mask]) * weight
            loss_dfl = loss_dfl.sum() / target_scores_sum # 归一化
        else:
            loss_dfl = torch.tensor(0.0).to(pred_dist.device)

        return loss_iou, loss_dfl
```

- dfl损失计算

实例设定

假设真实坐标值为 $y = 5.3$，离散区间间隔为1，相邻离散点为 $y_i = 5$ 和 $y_{i+1} = 6$。模型预测这两个点概率分别为：

$$\mathcal{S}_i = 0.4, \quad \mathcal{S}_{i+1} = 0.3$$

其他14个离散点的概率总和为 $1 - 0.4 - 0.3 = 0.3$，但DFL仅优化相邻两点。

损失计算步骤

1. **计算权重系数**

   真实值与离散点的距离权重为：

$$\mathcal{S}_i^{\text{target}} = \frac{y_{i+1} - y}{y_{i+1} - y_i} = \frac{6 - 5.3}{1} = 0.7$$

$$\mathcal{S}_{i+1}^{\text{target}} = \frac{y - y_i}{y_{i+1} - y_i} = \frac{5.3 - 5}{1} = 0.3$$

2. **代入DFL公式**

   损失值为：

$$\mathbf{DFL} = -\left(0.7 \cdot \log(0.4) + 0.3 \cdot \log(0.3)\right)$$

3. **计算对数项**

$$\log(0.4) \approx -0.9163, \quad \log(0.3) \approx -1.2039$$

4. **最终结果**

$$\mathbf{DFL} = -\left(0.7 \times (-0.9163) + 0.3 \times (-1.2039)\right) = -\left(-0.6414 - 0.3612\right) = 1.0026$$

```python
class DFLoss(nn.Module):
    """Criterion class for computing Distribution Focal Loss (DFL)."""

    def __init__(self, reg_max: int = 16) -> None:
        """Initialize the DFL module with regularization maximum."""
        super().__init__()
        self.reg_max = reg_max

    def __call__(self, pred_dist: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
        """Return sum of left and right DFL losses from https://ieeexplore.ieee.org/document/9792391."""
        target = target.clamp_(0, self.reg_max - 1 - 0.01)
        tl = target.long()  # target left
        tr = tl + 1  # target right
        wl = tr - target  # weight left
        wr = 1 - wl  # weight right
        return (
            F.cross_entropy(pred_dist, tl.view(-1), reduction="none").view(tl.shape) * wl
            + F.cross_entropy(pred_dist, tr.view(-1), reduction="none").view(tl.shape) * wr
        ).mean(-1, keepdim=True)
```